

ניתוח ועיצוב מערכות מידע

תרשים מחלקות — Class Diagram

הרצאה 6-7

עודכן: 30.08.2026

1 יסודות — מחלקה ותרשים

2 אנטומיה של מחלקה

3 קשרים בין מחלקות

4 מניתוח לעיצוב טוב

5 תרגול וסיכום

מה קורה בפרויקט בנקודה הזו

סוף שלב הניתוח — מסירה מצוות הניתוח העסקי לצוות הטכני

צוות ניתוח עסקי (BA)

סיים לענות על מה המערכת צריכה לעשות — ומוסר חבילת תוצרים:

דרישות SMART
הרצאה 3

קטלוג חוקים עסקיים (BR-n)
הרצאה 3 · חוק שטרם הוחלט = החלטה שעוד חסרה לעיצוב

תרחישי שימוש (Use Case)
הרצאה 4-5

תרשימי תהליך (BPMN)
הרצאה 2

דרישות לא-פונקציונליות
עומס, זמינות, אבטחה, ביצועים



מסירה

סקירה משותפת ואישור התוצרים — לא "זריקה מעבר לגדר". ה-BA נשאר זמין להבהרות לאורך העיצוב.

צוות טכני — עיצוב

עונה על איך המערכת תיבנה. לא איש אחד: כמה בעלי תפקיד עובדים במקביל על אותו קלט.

היום

מודל הנתונים

איזה מידע המערכת מחזיקה ואיך הוא מאורגן — תרשים מחלקות מנתח מערכת (System Analyst)

מסד הנתונים

טבלאות, מפתחות, אינדקסים, גיבוי ושחזור
DBA · מהנדס נתונים

ממשק המשתמש

מסכים, זרימות עבודה, נגישות ושפה ויזואלית
UX / UI

ארכיטקטורה ותשתית

שכבות, אינטגרציה למערכות קיימות, אבטחה, עמידה בעומס
ארכיטקט · DevOps

כל הכיסאות עובדים על אותה חבילת דרישות — היום אנחנו יושבים באחד מהם: מודל הנתונים.

Business Analyst = BA · אנליסט/ית עסקית · SMART · Business Process Model and Notation = BPMN · מחזור פיתוח תוכנה · Software Development Life Cycle = SDLC · ספציפי, מדיד, בר-השגה, רלוונטי, עקיב · Specific, Measurable, Achievable, Relevant, Traceable =

עיצוב

היכן אנחנו ב-SDLC

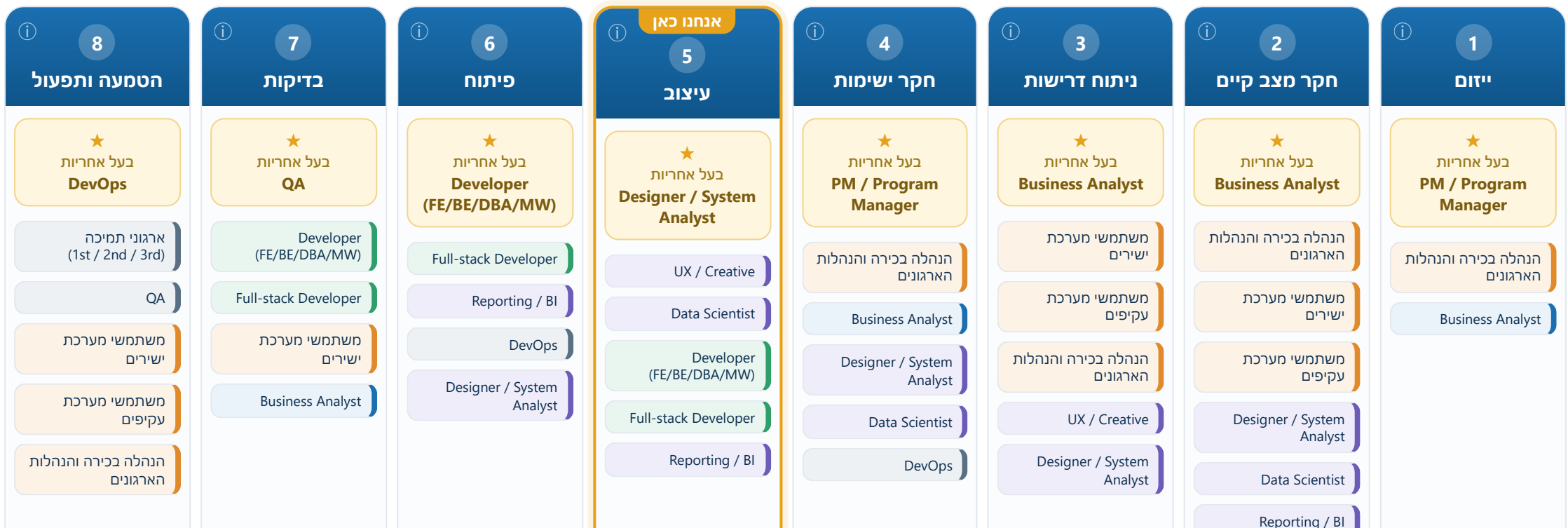
שלב עיצוב — כל שלב במחזור החיים ובעלי התפקידים המעורבים בו · כאן עוברים ממה לאיך

שלבי ה-SDLC — מי מוביל ומי משתתף בכל שלב

מיפוי בעלי התפקידים והאקטורים לאורך מחזור החיים של מערכת המידע

← הזמן מתקדם מימין לשמאל

לחצו על שלב או על תפקיד לקבלת הסבר קצר.



עיצוב

מחלקות ואובייקטים

Class = תבנית · Object = מופע

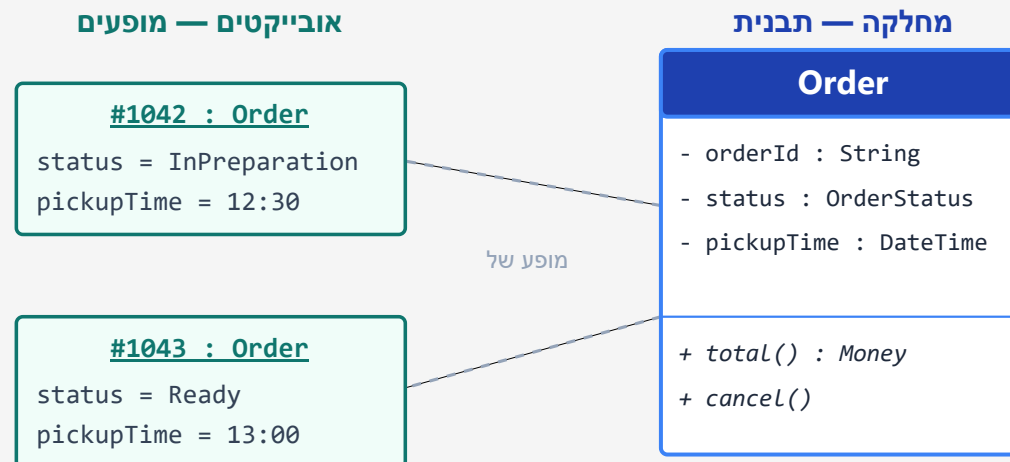
מחלקה = תבנית

הגדרה כללית של קבוצת אובייקטים בעלי אותן תכונות ופעולות. Class — כמו טופס ריק, נכתב פעם אחת.

אובייקט = מופע

Object קונקרטי בודד של המחלקה, עם ערכים ממשיים — הזמנה מסוימת של סטודנט מסוים.

מחלקה אחת — Order — הרבה מופעים



בעבודה עם AI

אפשר לבקש מ-AI למנות מתוך המפרט את המחלקות המועמדות (התבניות) — לא את האובייקטים הבודדים.

התרשים ממדל מחלקות (תבניות) — לא אובייקטים בודדים.

מהו תרשים מחלקות

המבנה הסטטי של המערכת — domain model

הגדרה

המבנה הסטטי של המערכת: המחלקות שהמערכת מנהלת והקשרים ביניהן. תרשים תרחישי שימוש מתאר התנהגות (מה) — תרשים המחלקות מתאר מבנה ונתונים (איך בנוי).

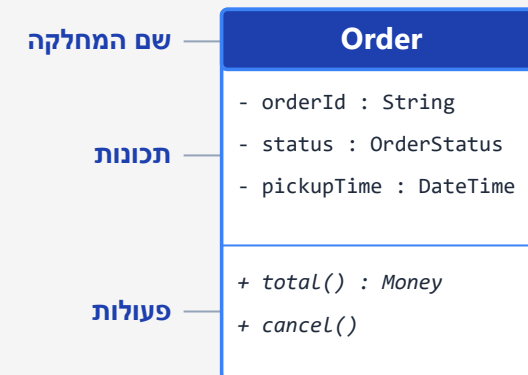
למה זה שימושי

- אוצר מילים משותף — שפה אחת לכל בעלי העניין על המחלקות של המערכת.
- השלד שממנו נגזרים בסיס הנתונים והקוד — גזירה, לא העתקה: המודל קובע איזה מידע קיים; איך הוא נשמר בפועל זו החלטת עיצוב בפני עצמה.

שכבת הנתונים 🗄️

זה גם מה ש-AI קורא כדי לייצר את שכבת הנתונים והפעולות — מודל ברור פנימה, קוד נכון החוצה.

שלושה חלקים לכל מחלקה



תוצר מרכזי של שלב העיצוב: מ"מ המערכת עושה" אל "איך בנוי המידע שלה".

use cases מ-למודל מחלקות

ה-AI מנסח מודל בשניות — ואנחנו מאתגרים אותו

מדביקים ל-AI את תרחישי השימוש והדרישות (הרצאה 4-5) — ומקבלים טיוטת מודל מחלקות עם קשרים ראשוניים, מיד.

⚠️ נראה שלם — אבל ה-AI נופל שוב ושוב

- ✗ הופך דוח מכירות / קבלה למחלקה — זה פלט מחושב שלא נשמר (מבחן ההתמדה — נרחיב בהמשך ההרצאה).
- ✗ ממציא מחלקות שהוצאו מהגבול — משלוח (מחוץ ל-scope).
- ✗ מנחש קרדינליות במקום לברר אותה.
- ✗ בונה תת-מחלקות שאין להן הצדקה — "הזמנה ממתינה" ו"הזמנה מוכנה" אינן סוגים של הזמנה, אלא סטטוס של אותה הזמנה.

🤖 מה ה-AI מפיק

רשימת מחלקות מועמדות מתוך אותה דוגמה רצה:

סטודנט הזמנה שורת הזמנה פריט תפריט תשלום עמדה קפיטריה

ומעליהן — הצעת קשרים וקרדינליות ראשונית. נראה שלם ומשכנע.

ההרגל שנקבע כאן

המודל הוא משטח האימות שלך. על כל טיוטה שאל: מה הומצא? מה חסר? מה הונח?

● עיצוב

מחלקה — התיבה ושלושת החלקים

כל מחלקה = מלבן ובו שלושה תאים: שם · תכונות · פעולות

מחלקה = תיאור של קבוצת אובייקטים בעלי תכונות, פעולות וקשרים דומים.

Order

- orderId : String
- pickupTime : DateTime
- status : OrderStatus

+ total() : Money
+ cancel()
+ addItem(item : MenuItem, qty : int) : OrderLine
+ getStatus() : OrderStatus
+ setStatus(newStatus : OrderStatus)

שלושת החלקים

- 1 שם המחלקה — בראש התיבה, על הרקע הכחול
- 2 תכונות (Attributes) — הנתונים שהמחלקה מחזיקה על כל אובייקט
- 3 פעולות (Operations) — מה המחלקה יודעת לעשות, נכתבות בכתב נטוי

שם המחלקה

איך נראה שם תקין — ולמה הוא תמיד באנגלית

שם מחלקה הוא מזהה — הוא יהפוך לשם טבלה ולשם מחלקה בקוד. לכן יש לו כללי כתיבה, והוא לא טקסט חופשי.

אותו רעיון, שני ניסוחים

למה	✓ תקין	✗ שגוי
ביחיד — המחלקה היא התבנית לאובייקט אחד	Order	orders
בלי רווח; כל מילה נפתחת באות גדולה	MenuItem	menu-item
בלי קו תחתון או מקף	OrderLine	order_line
שם עצם — פעולה היא operation בתוך המחלקה	Order	PlaceOrder
מזהים נכתבים באנגלית — אין שמות טבלאות ומחלקות בעברית	Order	הזמנה

✓ אותו היגיון לתכונות ולפעולות
גם הן באנגלית — רק שהן נפתחות באות קטנה (lowerCamelCase): orderId · pickupTime · addItem() · getStatus()
האות הראשונה היא מה שמבדיל מחלקה מתכונה.

כללי מתן שם

- מתחיל באות גדולה
- UpperCamelCase — כל מילה באות גדולה, בלי רווח ובלי מקף
- ביחיד — מחלקה מתארת אובייקט אחד, לא אוסף
- שם עצם, לא פעולה
- באנגלית — תמיד, גם כשכל השאר בעברית

💡 למה זה לא קפריזה
התרשים נקרא גם על ידי מפתח או AI שלא דובר עברית — ובסוף הוא הופך לשם טבלה ולשדה ב-API.

מדברים על ההזמנה בעברית — וכותבים Order בתרשים. זו העבודה בשטח.

Attributes — תכונות

הנתונים שהמחלקה מחזיקה על כל אובייקט

```
- name : Type = default {constraint}
```

רק השם חובה — כל שאר המרכיבים לפי הצורך

Order

- orderId : String
- createdAt : DateTime
- pickupTime : DateTime
- pickupCode : String
- status : OrderStatus

+ total() : Money ...

מרכיבי התכונה

- שם — באות קטנה, **camelCase**

- טיפוס (**dataType**) — אחרי הנקודתיים: **String**, **DateTime**, **Money**...

- ברירת מחדל — ערך התחלתי אחרי =

- אילוצים — תמיד בסוגריים מסולסלים { }

- נראות — - / # / + (בהרחבה בשקופית הנראות)

UML מול שפת תכנות

התרשים הוא הסימון — השפה היא המימוש, והמעבר ביניהם אינו אוטומטי

כל מה שנלמד בהרצאה הוא תקן UML — ולא שייך לאף שפה

UML נוצר כדי שמהנדס יוכל לתאר מערכת בלי להתחייב ל-Java, ל-Python או ל-SQL. לכן שם הטיפוס בתרשים הוא שם שאנחנו בוחרים, לא מילה שמורה בשפה.

איפה השפה לא עוקבת

Money אינו טיפוס בשום שפה

זה טיפוס תחום שאנחנו הגדרנו. מותר וכדאי — התרשים מדבר בשפת העסק.

הורשה מרובה

קיימת ב-UML וב-C++ ;ב-Java וב-C# רק דרך interface.

הרכבה וצבירה

אין להן מילה שמורה באף שפה — הן הופכות לכלל מחיקה ולבדיקות בקוד ובמסד הנתונים.

אותו מרכיב במודל, חמישה יעדים

SQL	Python	Java	C# הפרויקט שלכם	בתרשים (UML)
VARCHAR(n)	str	String	string	String
TIMESTAMP	datetime	LocalDateTime	DateTime	DateTime
NUMERIC(10,2)	Decimal	BigDecimal	decimal	Money
View / הרשאות	מוסכמה בלבד — אין אכיפה	private	private	- private
טבלה עם מפתח זר	list[Phone]	List<Phone>	List<Phone>	Phone [0..*]

השורה האחרונה היא קשר ולא תכונה — ולכן הריבוי בה חוקי. בקוד הוא הופך לאוסף של אובייקטים, ובמסד הנתונים לטבלה עם מפתח זר — אין שם רשימה בכלל.

המודל לא נכתב עבור שפה מסוימת — אל תצמצמו אותו למגבלות של שפה בשלב הניתוח.

מזהים — למי יש, ולמי אין

מזהה נכנס לתרשים רק אם העסק משתמש בו — לא כי הטבלה צריכה מפתח

המבחן: האם מישהו מחוץ למערכת מפנה לדבר הזה בעצמו? אם אומרים בקול "הזמנה 1042" — יש מזהה. אם אומרים "השניצל שבהזמנה 1042" — אין.

✗ אין מזהה משלו

OrderLine — אף אחד לא אומר "שורה 3". אומרים "השניצל בהזמנה 1042"

Station — מזהה בתוך הקפיטריה שלה: "העמדה החמה"

Cafeteria — מזהה בשם ובמיקום

ולא ממציאים להן מזהה רק כדי שיהיה.

✓ יש מזהה עסקי

Student · studentId — מודפס על כרטיס הסטודנט

Order · orderId, pickupCode — הסטודנט מקריא אותו בדלפק

MenuItem · itemId — קוד הפריט בקטלוג של הקפיטריה

Payment · paymentId — מופיע בקבלה ובאישור החיוב

⚠ ומפתח זר אינו תכונה

אל תוסיפו studentId ל-Order. מי הזמין — זה הקשר בין המחלקות (חלק ג'). המפתח הזר מופיע רק בטבלה.

🗄 אז איך מעדכנים אותן במסד הנתונים?

לאובייקט בזיכרון יש זהות בחינם — ההפניה עצמה. לשורה בטבלה אין: כדי לעדכן צריך לדעת לאן לפנות.
כדי להוסיף שורה לא צריך מזהה: כדי לעדכן או למחוק — כן. לכן למחלקה בלי מזהה עסקי המסד מוצא מפתח משלו — לרוב הצירוף של הקשרים שלה, ולפעמים מפתח שהוא ממציא — והוא נשאר במסד, לא בתרשים.

בזיכרון לאובייקט יש זהות בחינם; לשורה במסד צריך כתובת — ומשם נולד המזהה.

שתי דוגמאות — מהמחלקה לטבלה

אותה שאלה על שתי מחלקות: מאיפה מגיע המפתח של הטבלה?

OrderLine X — אין לה מזהה

OrderLine
- quantity : int
- unitPrice : Money
- notes : String
אין שורת מזהה — וזה תקין

↓
הופכת לטבלה

OrderLines	
orderId	מפתח ראשי מורכב + זר
itemId	מפתח ראשי מורכב + זר
quantity · unitPrice · notes	

לטבלה חייב להיות מפתח כדי שאפשר יהיה לעדכן שורה — והוא מגיע משני הקשרים שהשורה כבר מחזיקה: איזו הזמנה, איזה פריט. הצמד הוא המפתח, ושום דבר לא חוזר לתרשים: שם הקשרים כבר מצויירים.

Order ✓ — יש לה מזהה עסקי

Order
- orderId : String
- pickupCode : String
- status : OrderStatus

↓
הופכת לטבלה

Orders	
orderId	מפתח ראשי
pickupCode	
status	
studentId	מפתח זר — רק כאן

המזהה כבר היה בתרשים, והוא פשוט הופך למפתח הראשי. לא המצאנו כלום. studentId לעומת זאת לא בתיבה — בתרשים הוא הקשר לסטודנט.

דוגמאות ואילוצים

איך מפרטים ערך של תכונה — ואיך כותבים עליו חוק

דוגמאות ערך

`price : Money` טיפוס בלבד

`price = 0` ערך התחלתי

`price : Money = 0` טיפוס + ברירת מחדל

`price {≥ 0}` אילוץ

אילוצים { }

אילוץ נכתב תמיד בסוגריים מסולסלים ומגדיר חוק על הערך:

`price {≥ 0} · quantity {> 0}`

יש גם מילות מפתח תקניות: `{readOnly}` · `{unique}` — לצד אילוץ חופשי משלכם.

אילוץ מורכב אפשר לתלות ב-Note (פתק):

`{ quantity > 0 }`

כשהמודל נפגש עם המציאות

טיפוס ואילוץ הם הבטחות — ואת אלה שנשברות משלמים בייצור

1 · הטיפוס הלא נכון



~~plateNumber : int~~

+ plateNumber : String

- אפסים מובילים נעלמים: 03-765-464 יהפוך ל-3
- בגרמניה יש אותיות: WOB ZK 295
- אף אחד לא מחבר שתי לוחיות רישוי

הכלל: אם לא מחשבים איתו — הוא לא מספר. גם ת"ז, טלפון ומיקוד הם String.

2 · אילוץ סגור מדי



~~machineColor = "White" {"White", "Black"}~~

- האילוץ קבע שיש שני צבעים בעולם
- ואז יצא הדגם הזה — והמערכת לא מסוגלת לקלוט אותו
- מי שמתקן את זה משנה קוד, לא נתונים

הכלל: רשימה שהעסק עשוי להרחיב אינה אילוץ — היא נתונים. אילוץ שומרים למה שבאמת לא ישתנה (≥ 0).

כל אילוץ שאתם כותבים — שאלו קודם: מה יקרה כשהמציאות תפר אותו?

Enumeration — רשימת ערכים סגורה

עד עכשיו כתבנו String, DateTime : — כאן מגדירים טיפוס משלנו, שהערכים החוקיים שלו ידועים מראש

אותו תנאי — שלוש דרכים לכתוב אותו

```
if (day == 1)
```

✗ מה זה 1? מספר קסם. צריך לזכור את הטבלה או לחפש אותה.

```
if (day == "Sunday")
```

✗ קריא — אבל אף אחד לא בודק. "Sundey" יתקמפל מצוין, וייכשל אצל הלקוח.

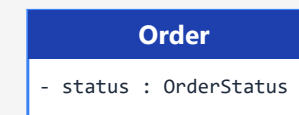
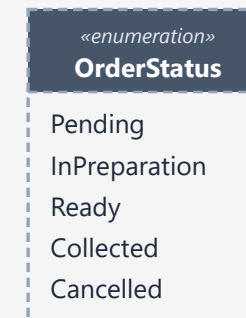
```
if (day == DayOfWeek.Sunday)
```

✓ קריא וגם נבדק. טעות כתיב לא מתקמפלת, וסביבת הפיתוח משלימה את הערכים החוקיים.

מתחת למכסה המנוע 💡

ב-C# הקומפיילר ממפה כל שם למספר — כותבים Sunday, ומשווים מספרים. לא בכל שפה כך:
ב-Java וב-Python ערך enum הוא אובייקט.

הסימון



ה-enum שבדוגמת הקוד
(מובנה ב-C#)

אין קו בין enum למחלקה!

טיפוס התכונה הוא כל הקישור

מה תקני ומה שלנו: תקני — «enumeration», רשימת הערכים, ובלי קו אל המחלקה. הרקע האפור והמסגרת המקווקוות הם מוסכמת התצוגה שלנו, לא דרישה — בכלי אחר אותו enum ייראה בתיבה רגילה.

שם קריא — בלי לוותר על בדיקה. זה כל הרווח.

enum או מחלקה?

השאלה המכריעה אינה כמה ערכים יש — אלא מי מנהל את הרשימה

שתי שאלות, בסדר הזה:

1. האם הקוד מתפצל לפי הערך (switch / if)? אם כן — הקוד חייב להכיר את הערכים.

2. מי בעל הרשימה — מפתח, או משתמש עסקי שרוצה לשנות אותה בעצמו?

enum — כשהרשימה של המפתח

- סגורה ויציבה — משתנה פעם בשנים, אם בכלל
- הקוד מתפצל לפיה: מה שקורה ל"מוכן" שונה ממה שקורה ל"בוטל"
- שינוי בה הוא שינוי קוד — וזה בסדר, כי ממילא צריך לכתוב את ההתנהגות החדשה

דוגמאות: OrderStatus · DayOfWeek · סוג תשלום

מחלקה — כשהרשימה של העסק

- העסק משנה אותה: מוסיפים, מוציאים משימוש, משנים שם
- לערכים יש נתונים משלהם — סדר תצוגה, פעיל/לא פעיל, תרגום, תיאור
- הקוד לא מתפצל לפיה — רק מציג, שומר ומסנן

דוגמאות: קטגוריות פריטים · סיבות ביטול · מגדר (ארגונים מרחיבים את הרשימה עם הזמן)

✗ ארבעה דברים שלא עושים להם enum:

מספרים

גודל 1-10 הוא Integer {1..10} size :
— לא One, Two, Three... ערך שסופרים
או מחשבים איתו הוא טיפוס מספרי עם אילוץ.

ערכים שהעסק מנהל

אם משהו בארגון מוסיף ערך ממסך — זו טבלה, כלומר מחלקה. enum משתנה רק בגרסה.

יותר מ-20 ערכים

כלל אצבע: אף קוד לא מתפצל ל-80 מקרים, ורשימה כזו תמיד תשתנה. זו טבלה.

ערכים עם נתונים משלהם

אם לכל ערך יש מחיר, תיאור או התנהגות שונה — זו מחלקה, ואם ההתנהגות שונה, הורשה (חלק ג').

⚠ ואי אפשר מסך ניהול ל-enum

ה-enum מהודר לתוך התוכנה. הוספת ערך = שינוי קוד ← בנייה ← בדיקות ← גרסה חדשה בייצור. מסך ניהול מחייב טבלה; טבלה מחייבת מחלקה.

אם משתמש צריך להוסיף ערך ביום שלישי בצהריים — זה כבר לא enum.

עיצוב

מגדר — המקרה שמלמד הכול

אותה תכונה, שלוש תשובות — אחת נשברת תמיד, ובין השתיים האחרות תלוי במערכת

שלוש אפשרויות — אחת פסולה, שתיים תלויות במערכת

- isMale : Boolean

✗ פסול תמיד — שתי אפשרויות, ומי שאינו אחת מהן פשוט לא ניתן להזנה. וגם: מה זה ?false

- gender : Gender «enumeration»

⚖ כשהמערכת רק מציגה או אוספת לסטטיסטיקה — הקוד לא מתפצל, והרשימה זזה אחת לשנים.

Gender — 0..1 (מחלקה + טבלה)

⚖ כשהרשימה מנוהלת מבחון — טפסים ורגולציה: מסך ניהול, סדר תצוגה, תרגום, השבתת ערך ישן.

enum למגדר — בעד ונגד

✓ בעד

- פשוט: אין טבלה, אין קשר, אין מסך ניהול לתחזק
- הערך מוגן — אי אפשר להזין ערך שאינו ברשימה
- קריא בקוד: Gender.Other, לא מחרוזת חופשית
- מספיק לחלוטין אם רק מציגים ומסכמים

✗ נגד

- ערך חדש = גרסה חדשה: קוד, בדיקות, פריסה
- אין סדר תצוגה, אין תרגום, אין תיאור לערך
- אין השבתה — ערך שיצא משימוש נשאר בקוד
- הבעלים האמיתי הוא רגולציה, לא המפתח

הראיות: הרשימה משתנה — ומגורמים שמחוץ לצוות

לראשונה באירופה: לא חובה להגדיר מין תינוק

מיום שישי תאפשר גרמניה לתינוקות שיוולדו עם מאפיינים של שני המינים להירשם בתעודת הלידה לא כזכר ולא כנקבה. עם כניסת החוק לתוקף יהיו בדרךון גרמני 3 שדות: M עבור זכר, F עבור נקבה X-4 עבור מין לא מוגדר

Recommend 142

פורסם: 11:15, 30.10.13 AFP

גרמניה, 2013 — נוסף ערך שלישי לטופס רשמי

פייסבוק — רשימה פתוחה, נערכת בלי לגעת בקוד

enum או מחלקה? — חידון

לחצו על כרטיס כדי להפוך אותו. קודם מחליטים בכיתה — אחר כך בודקים

בכל כרטיס — רשימת ערכים. שלוש השאלות: האם הקוד מתפצל? מי בעל הרשימה? והאם לערכים יש נתונים משלהם?

חד מלך חד מלך X

קרו, (בלעזאָן בליזשניצאָנעס חרובות) — אַלעמאָל אַ שפּאַץ שייַ יאָרצאָל
(יאָ קלען).

הקלחום-חם — שזה זוף אליבום? ידיד'ש אוסו
 מינזום ניא מיאוסט מאו. מיינץ מאשאל ילכ, חפסום
 חקלחום birthmtnymy69 חפסום ניא — מינזום

```

CreditCardPayment <- Payment
UniversityCardPayment

```

✓ **મનન**

דוקה. יזינאט מוי קיסוי אל' להנא קא — אטטט אונדז
הטעט הנאט וליטא, (החיתת חושט) לעטא

«enumeration» DayOfWeek
(built-in C#)

הערה

חדש מיכאל; ומצב החוץ קרוב היחסיפה לחג
איצו קר דוקה. איש-אל'יטשן האוה

```
CaToggeT
{ evitCazi ,reb70Y6Iqzib ,əman }
γToggeT 1 - *.0 meTInneM
```


יִשְׂרָאֵל חֲרוּקָשׁ הוּא — קָרָא לֹא יִפֹּל לְעֶמְתּוֹ זֹקֵף.
חֲמוּצָתוֹ יִלְעָב מִיַּחֲזָקָתוֹ. יִלְטוּבִיָּה חֲזוּשׁ.

Collected, Cancelled, Pending, InPreparation, Ready, «enumeration» OrderStatus

יזח

פונקטור $\rightarrow$ חיסול מילסטר זוקסו, זכאב חיווה

שי מיכאעל → מינאמאש האשם ANJ האחד לא
 תפוצה {מסמך, מסמך} הקלחה ← מינאמאש

שפוט

→ מונח (זיכר פ' ל' fi) זוכר מוציף מואצחח

הקלטת → פסוק מואצת היצא מנהל

 חברת צדקה

פחמון → הקייטסיטטס' מיססוא וא, מייצח רך
ר'ס

הקצתם → תלמוד חז"ל — מוצי
 { אֲשֶׁר } אֲנִי אֲשֶׁר

הקדמה

— לצפתה אל דוקה; ונזנה לא חוביס קיסוס קסטה
הקלחה $\Leftarrow$ הלפט $\Leftarrow$ לוהי קס. חוודתו חובש קר

```
0rder 0..* - 1..0 CancelationReason { name, isActive }
```

Operations — פעולות המחלקה

מה המחלקה יודעת לעשות — החלק השלישי בתיבה

התיבה — חלק הפעולות מודגש

Order
- orderId : String - status : OrderStatus - pickupTime : DateTime
+ total() : Money + cancel()

חלק שלישי בתיבה — נכתב בכתב נטוי

פורמט הפעולה

+ method(param : Type) : ReturnType

- שם הפעולה כמו תכונה — camelCase, מתחיל באות קטנה
- פרמטרים עם טיפוסים בסוגריים — (param : Type)
- טיפוס הערך המוחזר אחרי הנקודתיים — Return : (אם קיים)
- נראות בהתחלה — - / # / +
- הפעולות נכתבות בכתב נטוי — כך מבחינים אותן מהתכונות

+ markReady()

פעולה בעלת "תופעת לוואי"

+ cancel()

ללא ערך מוחזר — משנה מצב בלבד

+ total() : Money

מחזירה ערך — טיפוס Money

עיצוב

הפעולות נכתבות בכתב נטוי — כך מבחינים אותן מהתכונות בתיבה.

התרשים הוא תצוגה — לא המודל

מה שלא מופיע בתיבה — לא נעלם. הוא פשוט לא מוצג

פעולה בלי פרמטרים בתרשים — עדיין יש לה פרמטרים

בתרשים מראים מה שרלוונטי לדיון. המפרט המלא — טיפוסים, ברירות מחדל, אילוצים, פרמטרים וערכי החזרה — קיים במודל, גם כשהתיבה לא מציגה אותו. לחצו על מחלקה כדי לראות את המפרט המלא שלה.

Payment

אין לה בכלל תכונות ופעולות?

👉 לחצו לראות

MenuItem

- name
- price
- available

- + *updatePrice(p : Money) : void*
- + *isAvailable() : Boolean*

לתכונות אין טיפוסים?

👉 לחצו לראות

Order

- orderId : String
- pickupTime : DateTime
- status : OrderStatus

- + *addItem()*
- + *cancel()*
- + *total()*

לפעולות אין פרמטרים וערך מוחזר?

👉 לחצו לראות

מאוחסן או מחושב?

ההחלטה שקובעת אם ערך הוא תכונה או פעולה

מאוחסן ← תכונה

- unitPrice : Money

נשמר בפועל. יש לו מקור — מישהו הזין אותו, או המערכת קבעה אותו ברגע מסוים.

C#: { get; set; } במסד: עמודה

מחושב ← פעולה

+ total() : Money

לא נשמר בשום מקום. מחושב מהנתונים שכבר יש, ברגע שמבקשים אותו.

C#: Total() במסד: אין עמודה

⚠ הטעות הנפוצה

לאחסן ערך שאפשר לחשב — זה מקור אמת שני. מוסיפים שורה להזמנה, שוכחים לעדכן את המונה, והמודל סותר את עצמו.

זו גם טעות אהובה על AI: טיוטות נוטות להוסיף totalAmount ו-itemCount כשדות שמורים.

✓ מתי כן מאחסנים למרות שנראה מחושב

ערך של רגע מסוים — unitPrice בשורת ההזמנה: מחיר בתפריט משתנה, וההזמנה מלפני חודש חייבת להישאר נכונה. גם: ביקורת ורגולציה (הערך מוקפא כפי שהיה), וביצועים על נפחים גדולים — החלטה מודעת, לא שכחה.

השאלה האחת: אפשר לחשב אותו מחדש?

אם אפשר לחשב אותו ממה שכבר שמור, ותמיד לקבל אותה תשובה — אל תאחסנו. הוא פעולה.

אם אי אפשר — הוא עובדה בפני עצמה, ולכן תכונה מאוחסנת.

נריץ את זה על סכום ההזמנה: אפשר לחשב מהשורות ותמיד נקבל אותו סכום ⇒ לא מאחסנים ⇒ **total() : Money** +.

ועל מחיר היחידה בשורה: אי אפשר — המחיר בתפריט השתנה מאז ⇒ מאחסנים ⇒ **unitPrice : Money** -.

מה שאפשר לחשב — לא מאחסנים. מה שלא — זו עובדה, ומאחסנים אותה.

Visibility — נראות

מי מורשה לגשת לתכונה או לפעולה של המחלקה

הסימון בתחילת כל חבר (תכונה או פעולה) קובע מי רשאי לגשת אליו — מבפנים, מהיורשים, או מכל מקום.

סימון	נראות	משמעות	דוגמה מהקמפוס
+	Public — ציבורי	כל מחלקה אחרת יכולה לגשת	total() : Money + על הזמנה — כל מי שמשתמש בהזמנה יכול לחשב את הסכום
#	Protected — מוגן	רק המחלקה ותתי-המחלקות (היורשים)	validate() # על תשלום — זמין לתשלום בכרטיס ולתשלום באשראי, לא מבחוץ
-	Private — פרטי	רק המחלקה עצמה	pickupCode : String - על הזמנה — מוסתר מכל מחלקה אחרת

הרגל שכדאי לבדוק 🤖 **ה-AI נוטה לסמן הכל כ-Public.** בשלב הניתוח: נתונים רגישים פרטיים (-), פעולות שירות שהמערכת חושפת ציבוריות (+).

בניתוח נסמן נראות בסיסית; העיצוב המלא מתחדד בקוד.

מה אינו מחלקה

מבחן ההתמדה — הפילטר החד ביותר בבחירת המחלקות

האם המערכת שומרת ומנהלת מידע על זה לאורך זמן?

❌ לא — אינו מחלקה

פלט מחושב (נגזר בזמן אמת)

- דוח מכירות · קבלה · אישור הזמנה מחושבים מנתונים קיימים בכל פעם מחדש — אין רשומה עצמאית לנהל.

משתתף שאינו נשמר

- הספק ("הספק מספק סחורה") — מחוץ ל-scope; המערכת לא מנהלת עליו רשומות

✅ כן — זו מחלקה

- הזמנה — נוצרת, נשמרת ומתעדכנת
- פריט תפריט — מנוהל ע"י המנהל, נקרא ע"י הסטודנט
- תשלום — רשומה נשמרת לכל תשלום

המערכת מנהלת עליהן CRUD — לכן הן מחלקות.

⚠️ הטעות מספר 1 של ה-AI בתרשים מחלקות

ה-AI הופך כל דוח / קבלה / אישור למחלקה. הריצו את מבחן ההתמדה — ובדיקת הצלבה: מועמדת שאין לה קשר לאף תרחיש שימוש, כנראה אינה מחלקה.

מחלקה או לא? — חידון

לחצו על כרטיס כדי להפוך אותו. קודם מחליטים בכיתה — אחר כך בודקים

המבחן בכל אחד מהם: האם המערכת שומרת ומנהלת מידע על זה לאורך זמן?



המוכר → המזנה לא מספק חזש

הקלחת → נהם מיחובש מורומש מובותח חות



אל → מולצאן נפסדזש קר חיח

צ → רחם היילא זונח'ל ידע ארומשען

✓ 22

— החייתם חושש, ואולי, מעט! לחזק את צי מא
דיחי ששום. טקייסוא לא חשעל מיביי מיחזח
חחלחח וייזח אוח.

X 4

הוא נ"א. מש"ל לכל חומצות שדחם בשוחם
ט"ס חז — להחל הוא נ"א חומצ"ל



א' → חספסה ל' שזחח חזצויה

צ → חלצו חססו מ, סמו חזקיסל חזאע

✓ 22

לְאִשִּׁיטָהּ חַיִּינָהּ, חֶסֶד חֲלָפָהּ, חֶחֱחֶהּ
וְהִנֵּה חֲלָחֶהּ חֲחֶחֶהּ — חֲחֶחֶחֶהּ חֲחֶחֶהּ

X 24

אזרח חמשנש חם. שווייט שיחתה — ח'לום וז
ח'טוים א'ל, ח'טויה

X **מְזַחַח מְחַלְחֵל מֵאֵל**

«פאפאמאן געזעט»-C — וואו מ'רײַצט אַ
ו.א. מאַרק פֿאַר, מײַנענדיג וילאָ מײַנענדיג אַלע וואַנע

הצגה מקוצרת של המחלקה

כשהמיקוד הוא הקשרים — מצמצמים את התיבה

כשהדיון הוא על הקשרים בין המחלקות, אין צורך בכל הפרטים. מצמצמים את התיבה ומשאירים רק את מה שחשוב.

Order

שם בלבד

Order

```
- orderId : String  
- status : OrderStatus  
- pickupTime : DateTime
```

שם + תכונות

Order

```
- orderId : String  
- status : OrderStatus  
- pickupTime : DateTime
```

```
+ total() : Money  
+ cancel()
```

מלא — שם, תכונות ופעולות

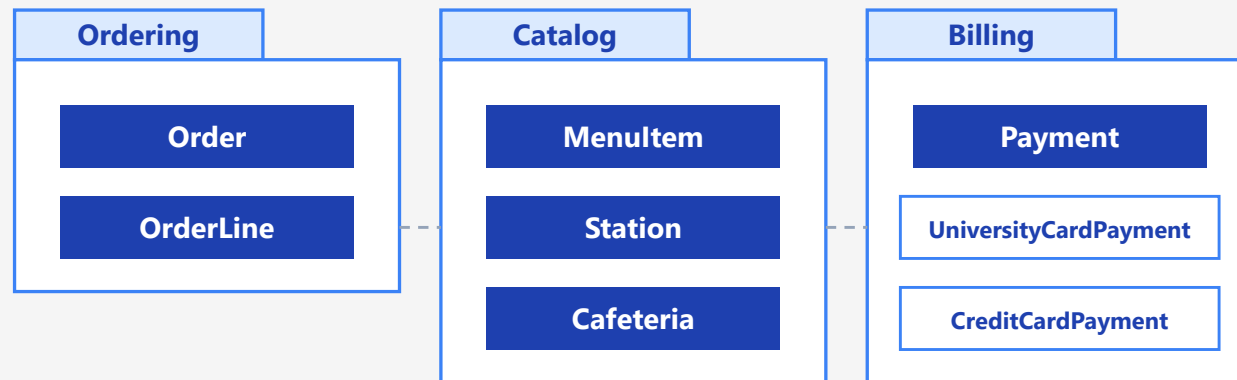
עיצוב

הראו את מה שחשוב לדיון — והשמיטו את השאר כדי למקד בקשרים. המחלקה עדיין מכילה את כל חבריה.

חבילות — Packages

איך שומרים על מודל גדול קריא: מקבצים מחלקות לפי נושא

מודל הקמפוס — מקובץ לשלוש חבילות



הקשרים בין המחלקות נשארים — החבילות רק מארגנות אותן

התיבות בהצגה מקוצרות — שם בלבד

שיוך במרחב שם: `Billing::Payment` · `Ordering::Order`

מה זו חבילה

- יחידת קיבוץ: מחלקות של אותו נושא נכנסות לתיקייה אחת
- מודל של 40 מחלקות הופך לתרשים של 4 חבילות — קודם רואים את המבנה, אחר כך נכנסים פנימה
- מרחב שם: `Ordering::Order` — שם המחלקה ייחודי בתוך החבילה שלה
- שם חבילה נכתב באנגלית, כמו שם מחלקה

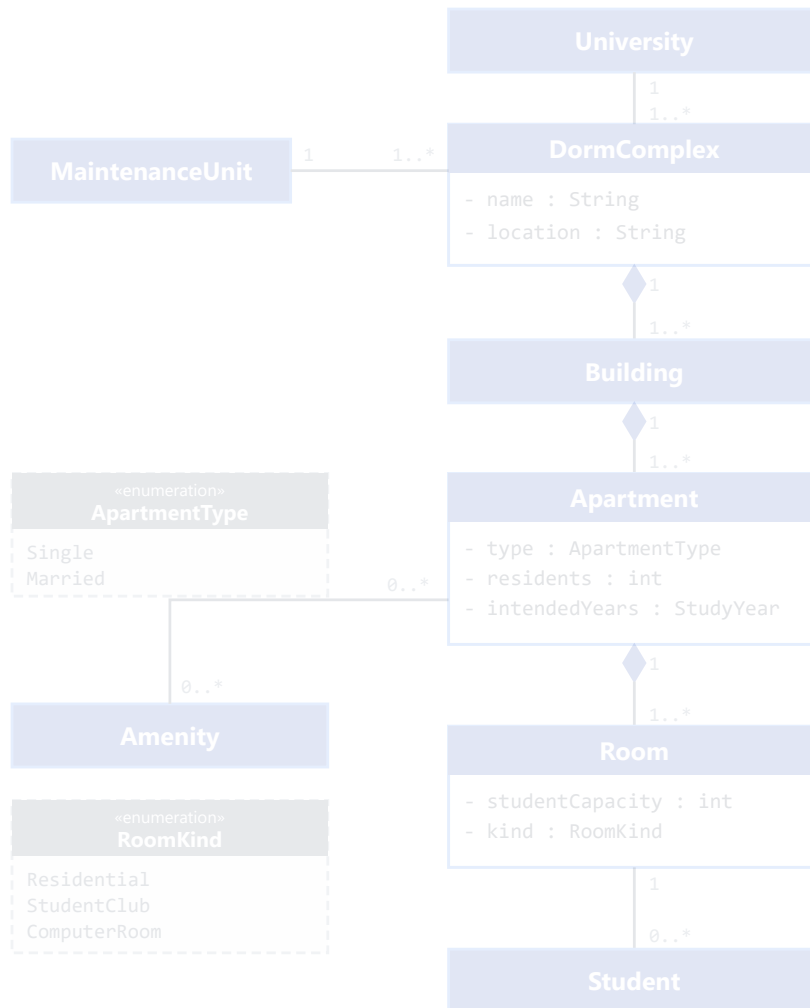
אתם כבר מכירים את זה 💡

ה-`import turtle` מקורס התכנות — חבילה שהמרצה שלכם כתב ואתם ייבאתם. אותו רעיון בדיוק, רק שכאן מציירים אותו.

בקורס הזה לא נשתמש בחבילות — מכירים את המונח, ולא נדרשים אליו בתרגילים.

תרגיל — זיהוי מחלקות

מערכת מידע למעונות: לחצו על כל מונח מודגש — קבלו את ההכרעה, והתרשים ייבנה תוך כדי



התרחיש

ל**אוניברסיטה** כמה **מקבצי מעונות** (מעונות ג', מעונות ד'...), כל מקבץ ב**מיקום** אחר בעיר. לכל המקבצים יש **אב בית** אחד שמטפל בתקלות.

בכל מקבץ יש **בנינים**, ובכל בניין **דירות** — מספר שונה בכל בניין. בחלק מהבנינים יש גם **חדרים שאינם למגורים** (מועדון סטודנטים, חדר מחשבים).

לדירה יש **סוג** (רווקים · נשואים), **כמות דיירים**, ורשימת **תנאים** — דוד חשמל, דוד שמש, כיריים גז, כיריים חשמל... כל דירה מיועדת ל**שנות לימוד** מסוימות.

בכל דירה כמה **חדרי מגורים**, ולכל חדר **כמות סטודנטים** משלו — יש חדרים ליחיד, לשניים וכו'. **סטודנטים** משובצים לחדרים על ידי משרד המעונות.

הערה: כמות הסטודנטים בחדר נקבעת על ידי **דיקאנט הסטודנטים**. היא קבועה בדרך כלל — אבל הדיקאנט יכול לשנות אותה (כך הפכו חדרי זוגות לחדרי יחיד).

■ מחלקה ■ תכונה ■ רשימת ערכים ■ מלכודת / לא מחלקה

מפת הקשרים

ארבעה סוגי קשר שבהם נשתמש — שניים מהם נפתחים בלחיצה: Association, על ארבע דרגותיו, ושלם-חלק על שני סימוניו



תלות

Dependency

קו מקווקו + חץ פתוח למערכת חיצונית
שאנחנו קוראים לה.

«external system»

דוגמה



אנחנו קוראים לסליקה — היא לא מחלקה שלנו

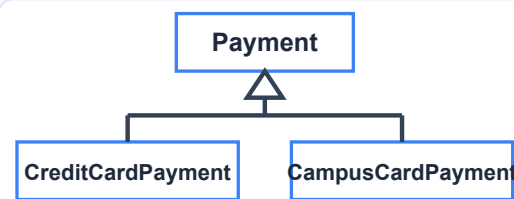


הכללה / הורשה

Generalization

▷ יחס is-a. רק כששונים בקשרים — לא בתכונות.

דוגמה



שונים בקשרים: אשראי ← חברת אשראי; כרטיס-אוניברסיטה ← חשבון סטודנט



שלם-חלק

Aggregation / Composition

מעוין בקצה השלם. השאלה: האם החלק חי בלי השלם?

שני סימונים — מדיניות שונה לכל אחד

צבירה ◊ — ליהוי בלבד, מציירים קו רגיל



הרכבה ◆ — מציירים, ויש לזה מחיר



לחיצה להרחבה 🔍

*..1 *..0

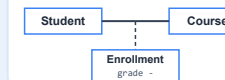
אסוציאציה

Association

קו מלא, בלי חץ — הקשר הבסיסי. קרדינליות בשני הקצוות.

ארבע דרגות של אותו קשר — מהפשוט למורכב

2 · מחלקת קשר



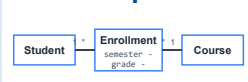
1 · קשר רגיל



n-ary · 4



3 · מחלקה מתווכת



לחיצה להרחבה 🔍

למעלה — איזה סוג קשר; בתוך הכרטיס הראשון — כמה מסובך הקשר הרגיל שלנו. בשקפים הבאים נצלול לכל אחד על מודל הקמפוס.

Association + קרדינליות

הקו שמחבר שתי מחלקות — וכמה מכל צד

דוגמה מהקמפוס



סטודנט אחד מבצע אפס או יותר הזמנות; כל הזמנה שייכת לסטודנט אחד בדיוק. קו מלא, בלי חץ.

❌ לא כותבים

- 1..20, 2, 0..3 — אין קרדינליות מספרית. מספר ספציפי הוא כלל עסקי שמשתנה $\Leftarrow$ {אילוץ} (שקף 32)
- M:N, N, M — סימון של מסדי נתונים, לא UML. כותבים 1..* / 0..*

מוסכמה של הקורס (לא של UML): קצה בלי קרדינליות נקרא כ-*. אז אם התכוונתם ל-1 — חובה לכתוב. ⚠️ ה-AI נוטה להשאיר קצה ריק או לנחש 1 — לבדוק שני קצוות מול המקור.

הכללים

- קו מלא — בלי חץ (חץ מכוון לא בשימוש על מודל התחום)
- קרדינליות בשני הקצוות — תמיד
- המשמעויות — וזה כל אוצר המילים:
 - 1 = בדיוק אחד
 - 0..1 = אפס או אחד (אופציונלי)
 - 1..* = אחד או יותר
 - 0..* = אפס או יותר (כל כמות)

קשר רקורסיבי (מחלקה לעצמה)



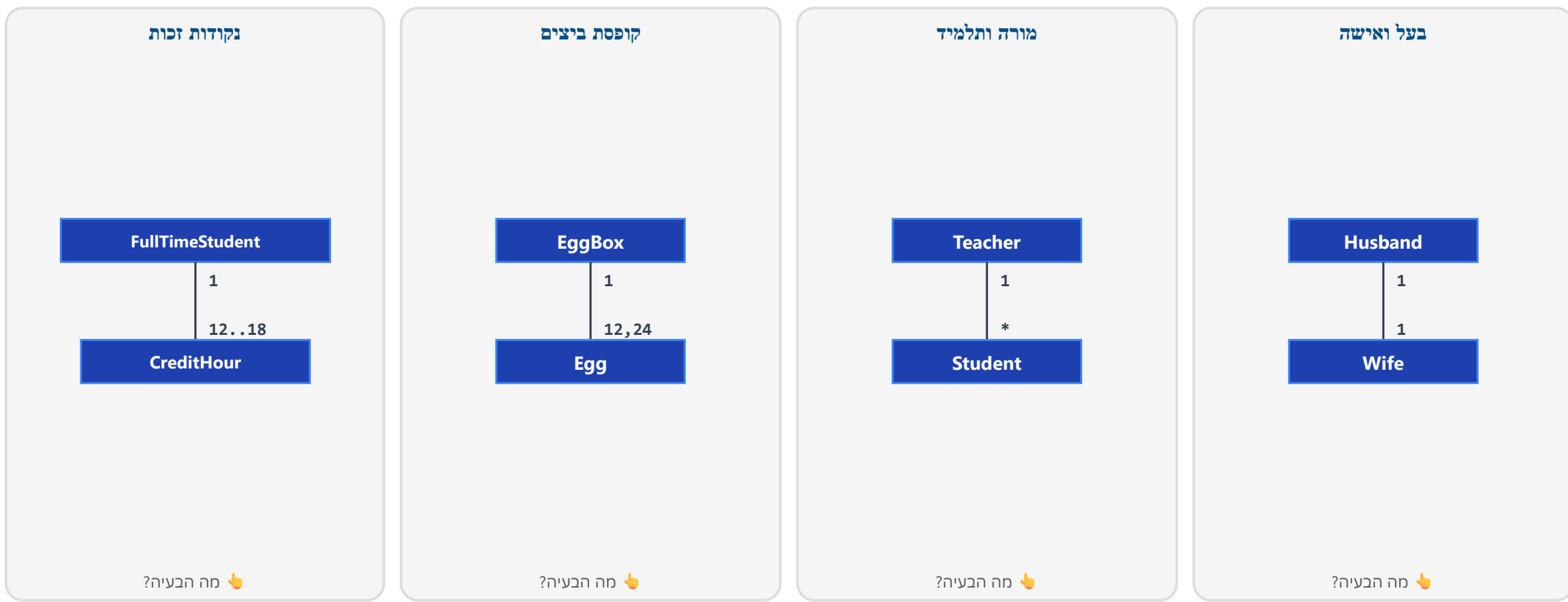
לקטגוריה יש הורה אחד ותת-קטגוריות משלה — אותה מחלקה בשני הקצוות. שמות תפקיד על הקצוות הם אופציונליים — כאן בלי הם לא היינו יודעים מי ההורה.

אילוץ {or}

הזמנה משויכת לתשלום בכרטיס אוניברסיטה {or} תשלום באשראי — אחד מהם, לא שניהם. מסמנים {or} על שני הקווים היוצאים.

הקרדינליות הקלאסית — ולמה רובה שגויה

לחיצה ראשונה — מה לא בסדר. לחיצה שנייה — מודל טוב



שלוש שאלות: 1. תמיד? או רק ברוב המקרים — ומה קורה במיעוט? 2. גם עם היסטוריה? ה-1 נכון היום, ולא לאורך זמן 3. לפי איזה חוק — ומי רשאי לשנות אותו?

קרדינליות היא החלטה עסקית — ולפעמים המספר שגוי כי המחלקה שגויה.

איזו קרדינליות כותבים — ומה כבר שייך למימוש

אנחנו בשלב העיצוב. התרשים שלכם הוא המפרט — והמימוש נגזר ממנו

✓ ארבע צורות — וזה הכול

1 0..1 1..* 0..*

כולן תשובות לשתי שאלות בלבד: האם זה יכול להיות אפס? והאם יכולים להיות יותר מאחד?

✗ מספרים ספציפיים — אלה לא קרדינליות

2 1..20 12..18 12,24

אף שפה ואף מסד נתונים לא אוכפים אותם, והם מקבעים חוק שמשתנה. מקומם באילוץ בפתק ובנתון — לא על הקו.

הקוד — Code

מה שה-AI גוזר מהתרשים שלכם. אתם לא כותבים אותו ולא מציירים אותו

```
class Order {
    Cafeteria cafeteria;           // 1 → שדה חובה
    List<OrderLine> lines = new(); // 0..* → רשימה

    bool CanAddLine() =>           // { } → בדיקה
        lines.Count < cafeteria.MaxOrderLines;
}

class OrderLine {
    Order order;                   // 1 → הפניה חזרה
}
```

בציוב (שלכם)	בקוד (של ה-AI)
0..*	List<OrderLine>
1	שדה חובה — אי אפשר ליצור בלעדיו
{ }	CanAddLine()

הקו נעלם והופך לשדות — בשני הכיוונים; הריבוי הופך ל-List; האילוץ הופך לפעולה. הכול נגזר מהתרשים היחיד שציירתם.

תרשים העיצוב — Design

התרשים היחיד שאתם מציירים ומוסרים



{ lines ≤ cafeteria.maxOrderLines }
the number is DATA on Cafeteria, not on the line
BR-7

איפה כותבים את החוק? בפתק (Note) שמחובר לקו הקשר, ובתוכו האילוץ בסוגריים מסולסלים. המספר עצמו הוא תכונה של Cafeteria — נתון שמנהל משנה, לא קרדינליות. והחוק עצמו כבר קיים בקטלוג מהרצאה 3 — לכן מפנים אליו (BR-7) במקום לשכתב אותו.

אתם מייצרים את תרשים העיצוב; ה-AI גוזר ממנו את המימוש — מה שלא אמרתם, הוא ימציא.

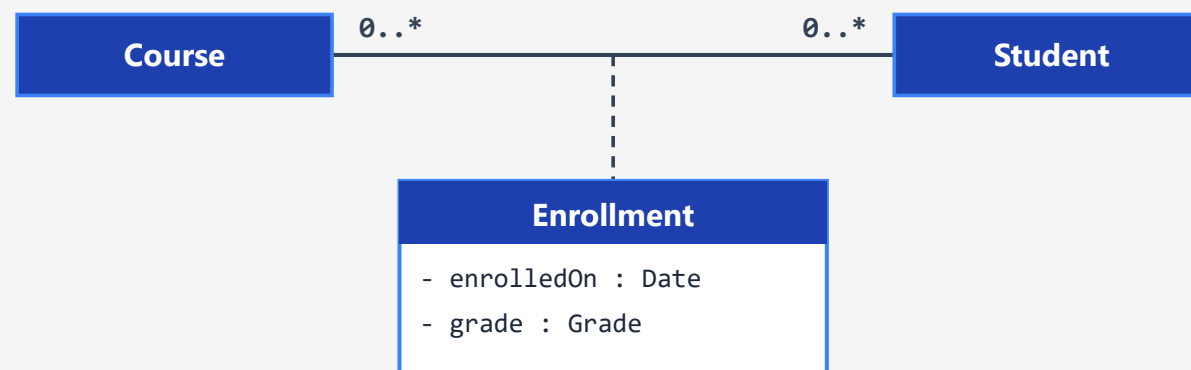
מחלקת קשר — כשלקשר עצמו יש נתונים

הציון אינו של הסטודנט ואינו של הקורס — הוא של הקשר ביניהם

איפה שמים את הציון?

× על הקורס? יש בו ציון אחר לכל סטודנט

× על הסטודנט? יש לו ציון אחר בכל קורס



✓ הציון קיים רק בזכות הצירוף של השניים — ולכן הוא נתלה על הקשר עצמו. המחלקה שנתלית כך נקראת מחלקת קשר.

איך מציירים

1 הקשר נשאר על מקומו — לא מוחקים אותו ולא מפצלים אותו.

2 קו מקווקו יוצא מאמצע קו הקשר אל קופסת המחלקה. בלי חץ ובלי יהלום.

3 הקרדינליות נשארת על קו הקשר; על הקו המקווקו אין קרדינליות — הוא לא קשר.

4 הקופסה היא מחלקה רגילה לכל דבר: שלושה מדורים, שם UpperCamelCase-2.

5 השם הוא שם עצם של הקשר — Enrollment, Membership, Employment — ולא פועל ולא "קשר".

1 למי שייך הנתון?

לא לצד אחד ולא לשני. "של מי הציון?" — של שניהם יחד. אם התשובה היא צד אחד, זו פשוט תכונה שלו.

2 כמה מופעים לכל צמד?

אחד לכל היותר. סטודנט אחד + קורס אחד = הרשמה אחת. הצמד הוא שמזהה את המופע — אין לו מזהה משלו.

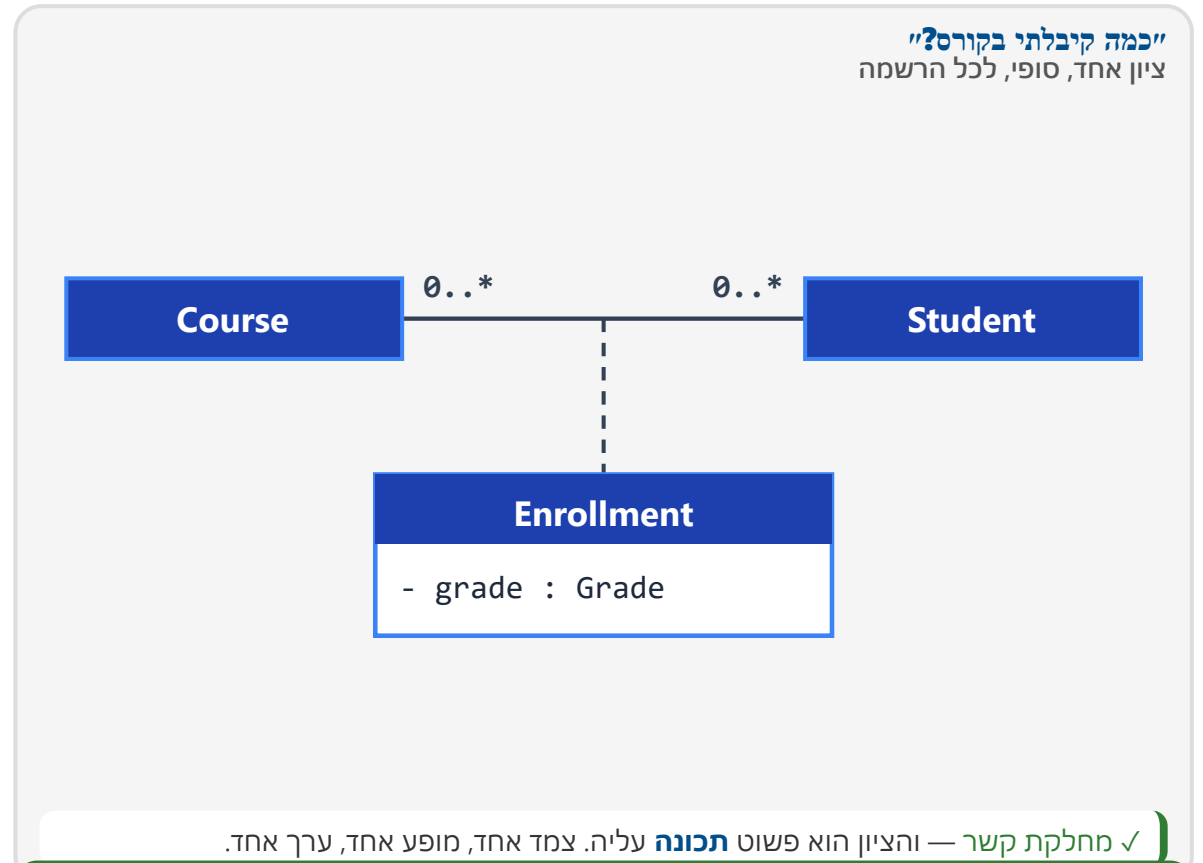
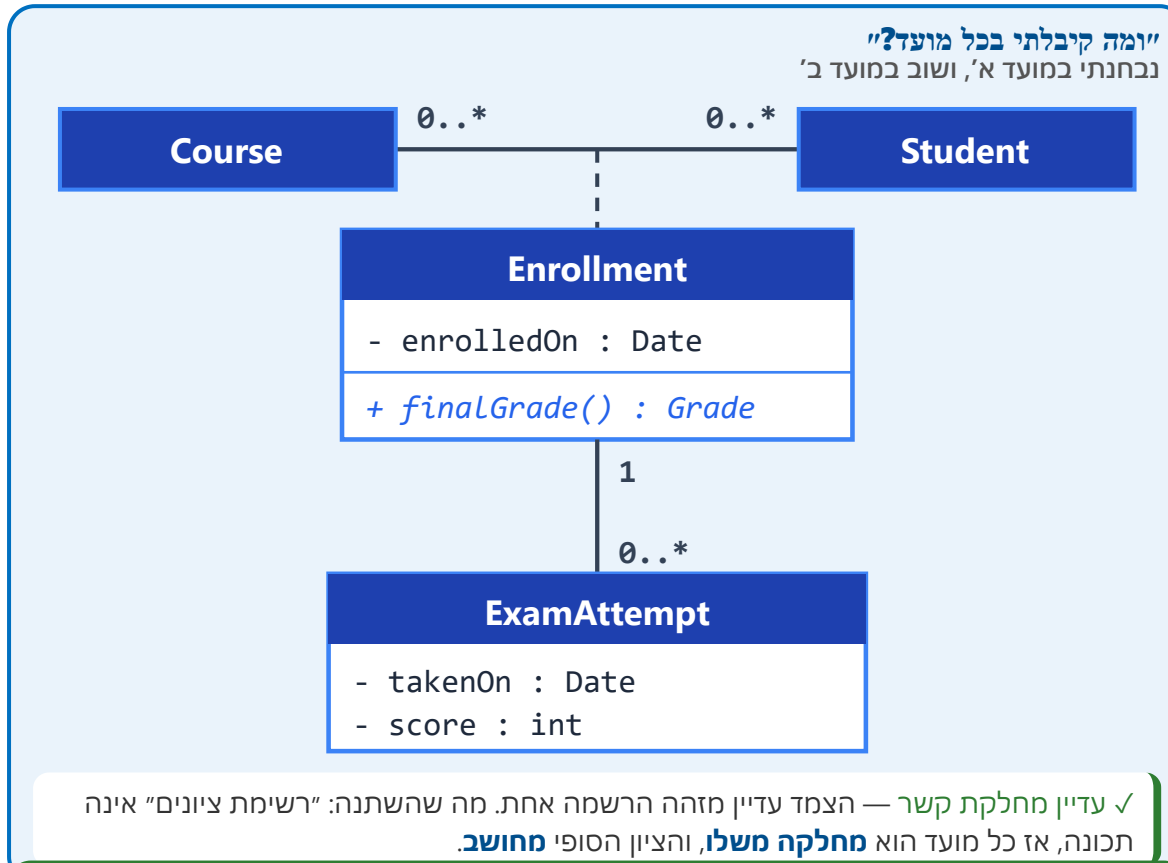
3 יש לו חיים בלי הקשר?

אין. הוא נולד עם הקשר ומת איתו. אף אחד בעסק לא אומר "הרשמה 4471" — אומרים "ההרשמה של דנה לקורס".

ובעיקר מבחן 2: צמד שיכול להופיע פעמיים — כבר אינו מחלקת קשר.

מחלקת קשר היא מחלקה לכל דבר

אותה דוגמה, שתי שאלות: "כמה קיבלתי בקורס?" מול "מה קיבלתי בכל מועד?"



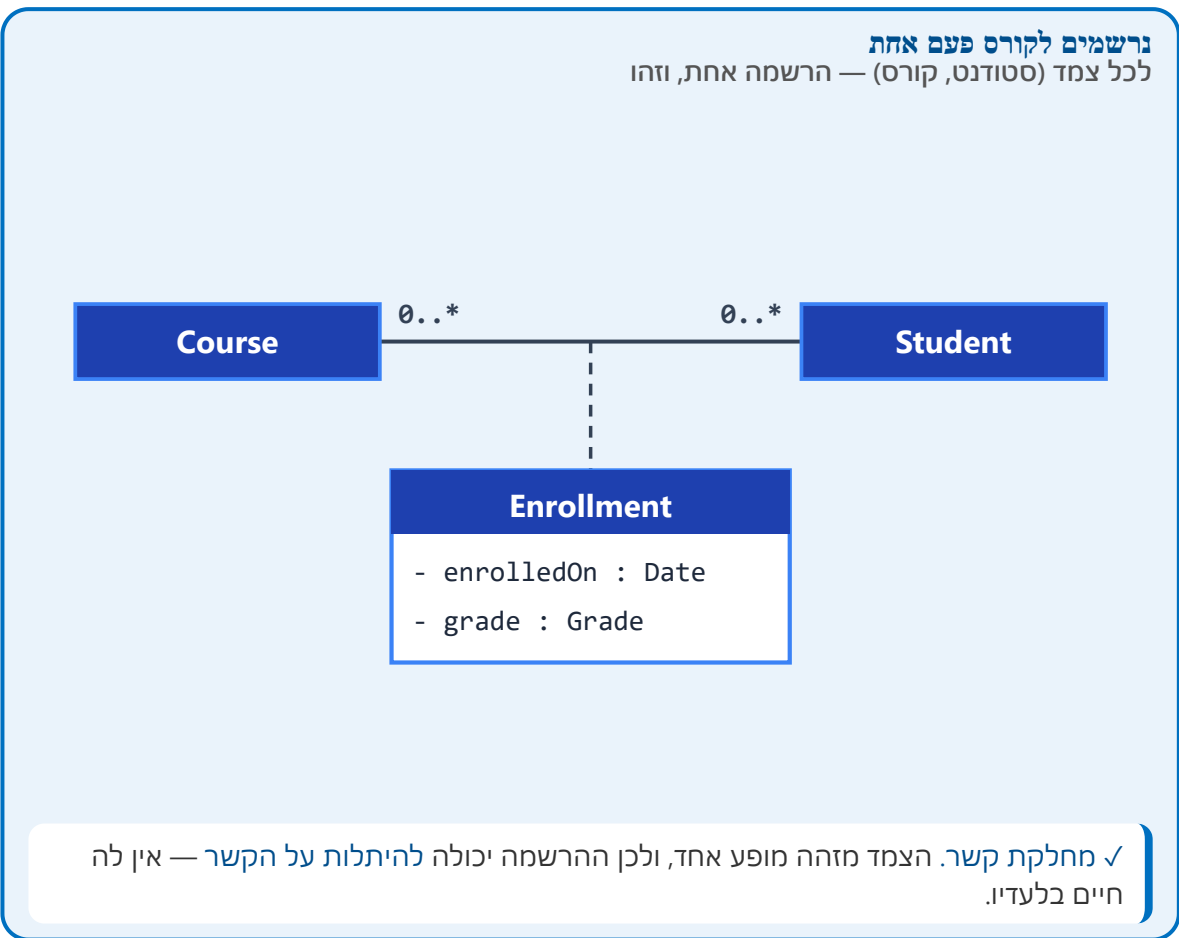
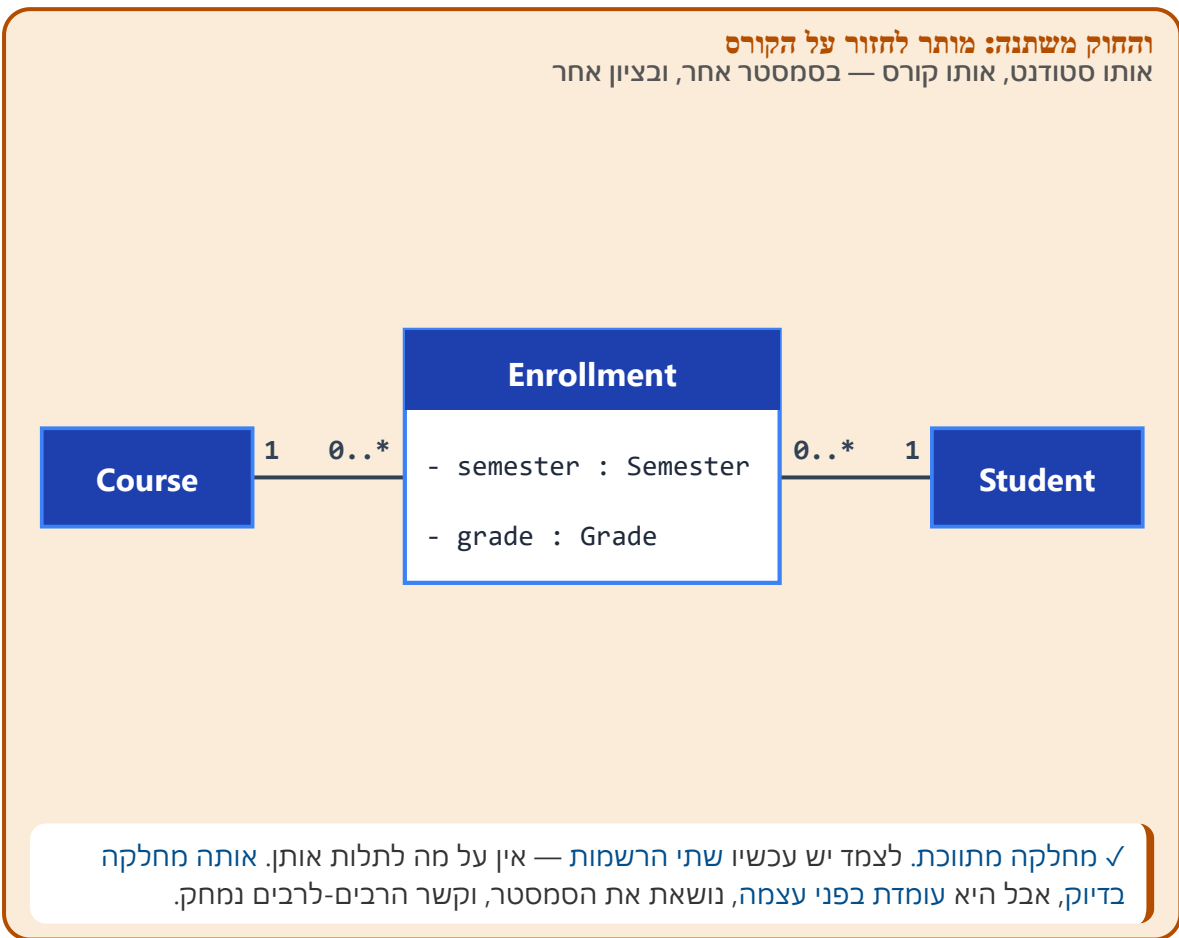
✓ **מותרים לה קשרים**
ExamAttempt תלוי **בהרשמה** — לא בסטודנט (באיזה קורס?) ולא בקורס (של מי?). רק להרשמה יש את שני הצדדים, ולכן היא הצד הנכון לקשר. **מה שעדיין אסור**: שני מופעים לאותו צמד.

✓ **מותרות לה פעולות**
אין ב-UML שום כלל שאוסר זאת — מחלקת קשר היא גם **מחלקה** וגם **קשר**, ולכן יש לה תכונות, פעולות והורשה. נדמה שאין, כי במודל תחום כמעט לא מציירים פעולות בכלל. וכאן חל בדיוק אותו כלל: **מחושב** ← פעולה.

מחלקת קשר אינה "חצי מחלקה" — מה שאין לה זהות ומחזור חיים משלה, וזה הכול.

מתי מחלקת קשר מספיקה — ומתי כבר לא

אותו סטודנט, אותו קורס, אותו ציון. מה שמכריע הוא חוק עסקי — האם אפשר להירשם פעמיים



ועוד שני סימנים שהמופע הוא "דבר" ולא "קשר": יש לו מצב — הרשמה שבוטלה, שורה שהוחזרה · ומישהו מזכיר אותו בשמו — "ההרשמה מסמסטר ב'". לקשר אין מצבים, ואף אחד לא מדבר עליו בפני עצמו.

דוגמה נוספת — שורת הזמנה בקמפוס

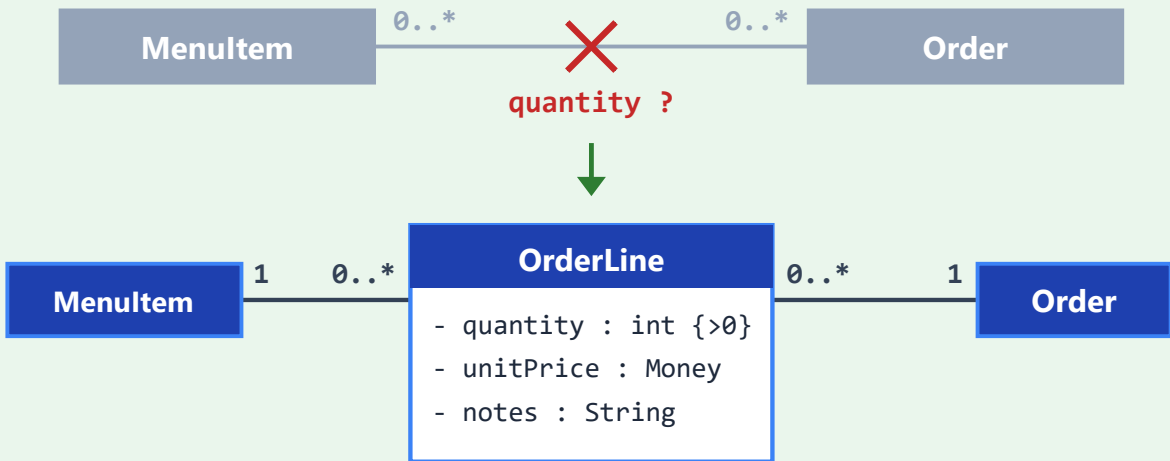
שניצל עם בצל ושניצל בלי, באותה הזמנה: אותו צמד — שתי שורות

אותה הכרעה, על המודל שלנו

- הצמד (הזמנה, פריט תפריט) חוזר: שתי שורות לאותו פריט, עם כמויות והערות שונות. אין על מה לתלות אותן — מחלקת קשר מחזיקה מופע אחד לכל צמד.
- ולשורה יש חיים משלה: אפשר לבטל שורה אחת בלי לבטל את ההזמנה, ולשנות בה כמות.
- ולכן OrderLine היא מחלקה מתווכת — היא מחליפה את קשר הרבים-לרבים בשתי אסוציאציות רגילות, ונושאת את מה שנולד מהקשר.

ואילו העסק היה אומר אחרת: "כל פריט מופיע בהזמנה פעם אחת, עם כמות" — אז הצמד היה מזהה, וזו הייתה מחלקת קשר. וכך בדיוק בנוי OrderItem בפרויקט לדוגמה של הקורס. אותה צורה בקוד — החלטת זהות אחרת.

מה שמצויר במודל הקמפוס

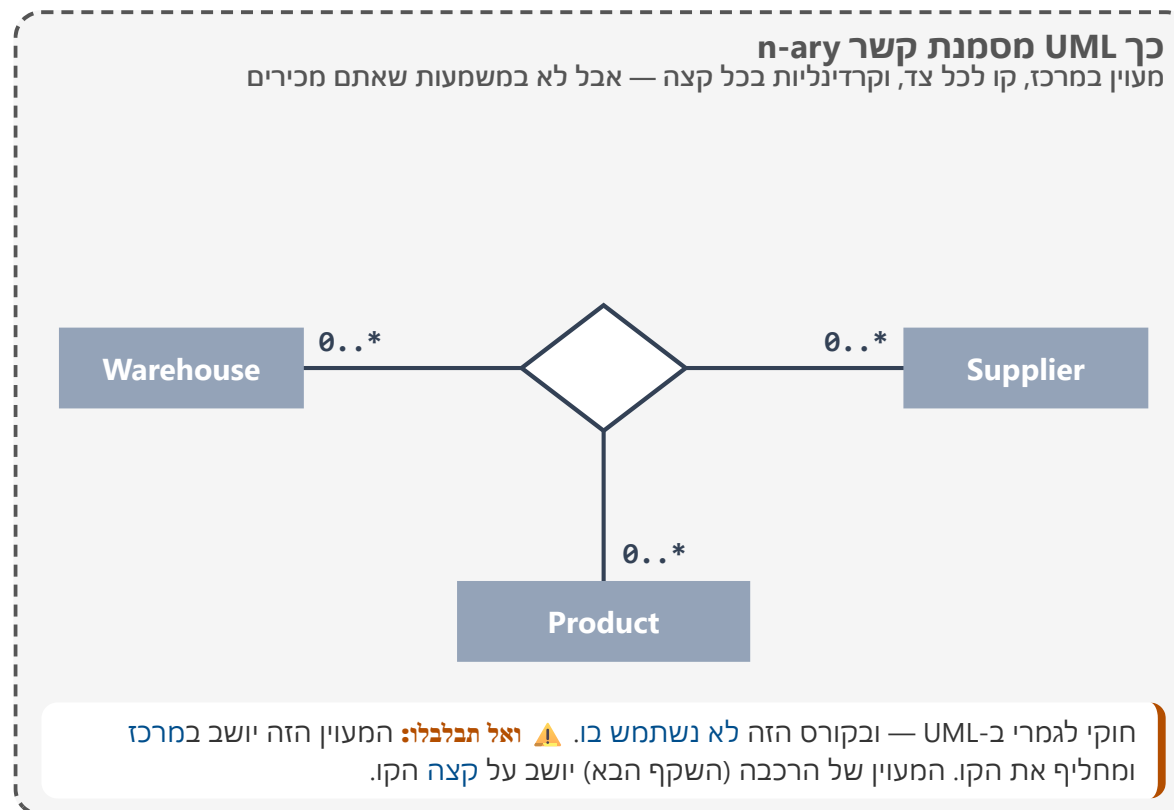
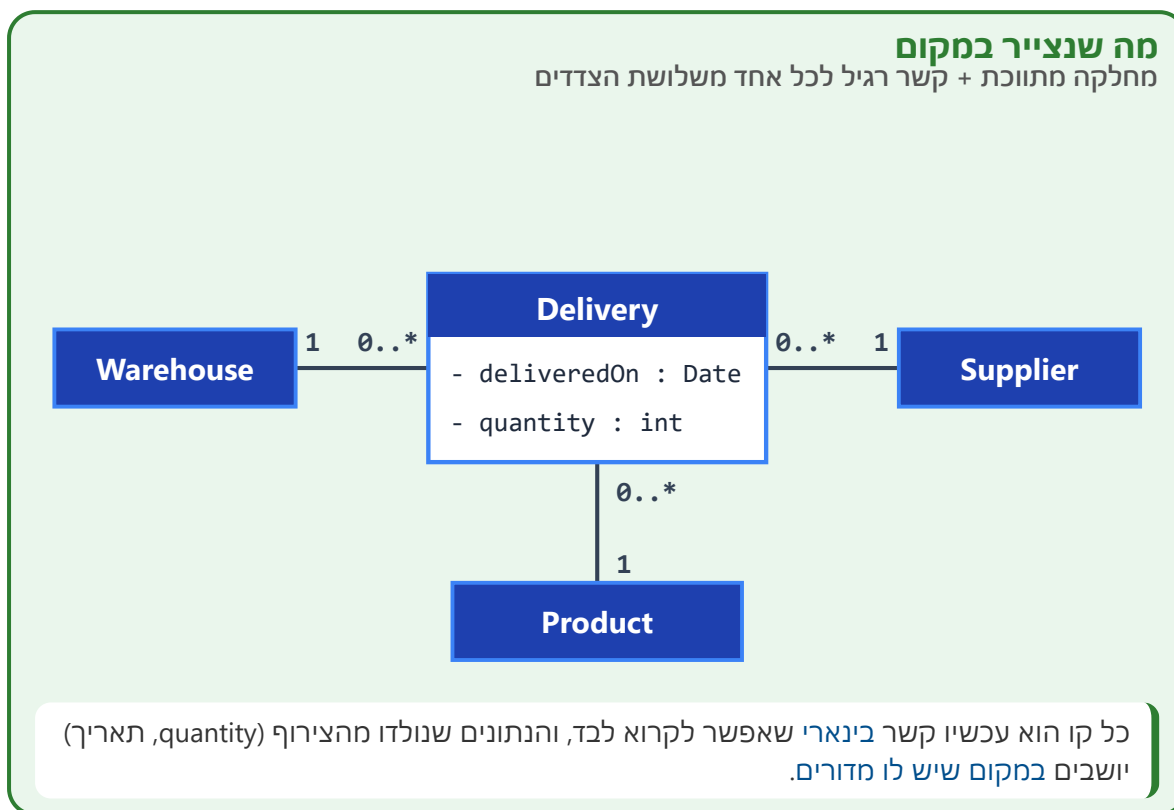


קשר הרבים-לרבים המקורי נמחק, ובמקומו שתי אסוציאציות רגילות ומחלקה באמצע. שימו לב: OrderLine אינה תלויה על קו — היא מחליפה אותו.

היבט	מחלקת קשר (Association Class)	מחלקה מתווכת (Mediating Class)
מבחן ההכרעה	הצמד מזהה מופע אחד לכל היותר	הצמד יכול לחזור, או שלמופע יש חיים משלו
סימון	הקשר נשאר; קו מקווקו אל אמצע הקו	מחליפה את הקשר בשתי אסוציאציות
במסד הנתונים	טבלה שמפתחה הוא הצמד (מפתח מורכב)	טבלה עם מפתח משלה, כמו כל מחלקה

קשר בין שלושה — הסימון שקיים, ולא נשתמש בו

”ספק X סיפק מוצר Y למחסן Z” — עובדה אחת שדורשת שלושה צדדים יחד



❌ אף אחד ב-**downstream** לא מייצג את זה

אין לזה מבנה בקוד ואין טבלה כזאת במסד. בסוף תמיד נוצרת טבלה עם מחלקה באמצע — אז עדיף לצייר אותה מלכתחילה.

❌ אין איפה לשמור נתונים

הכמות והתאריך שייכים לשלושת הצדדים יחד — ולמעוין אין מדורים. כדי לשמור אותם צריך ממילא מחלקה.

❌ הקרדינליות אומרת משהו אחר לגמרי

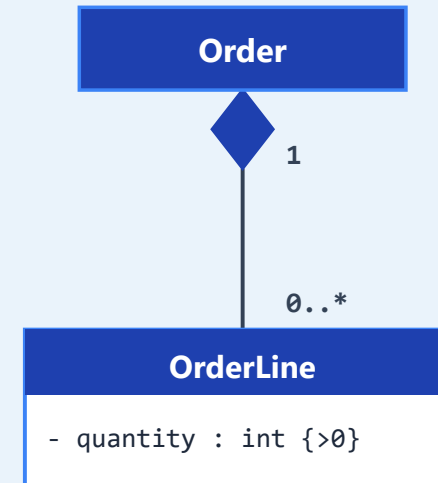
שימו לב שהמספרים בשני התרשימים שונים — וזו לא טעות: במעוין, 0..* בקצה של מוצר נספר לכל צירוף של ספק ומחסן. בפירוק, 1 אומר פשוט: כל אספקה היא של מוצר אחד. שתי שאלות שונות — ורק השנייה נקראת.

שלושה צדדים ומעלה ⇐ תמיד מחלקה מתווכת, ומכל צד קשר רגיל. את המעוין משאירים בתקן.

הרכבה ♦ — החלק חי ומת עם השלם

המעוין המלא יושב על השלם, והוא אומר משהו עם תוצאות — לא סתם "שייך ל..."

מה הסימון אומר



- 1 תלות קיומית: נמחקת ההזמנה — נמחקות איתה כל שורותיה. אין "שורה יתומה".
- 2 בעלים אחד בלבד: אי אפשר שאותה שורה תשתייך לשתי הזמנות. שיתוף שולל הרכבה — וזה המבחן שנופלים בו יותר.
- 3 ומה שמחייב את היצירה הוא המספר: ה-1 אומר ששורה לא נוצרת בלי הזמנה; ה-0..* אומר שהזמנה כן יכולה להיוולד ריקה.

מתי כן, ומתי בכלל לא

- ✓ הזמנה ♦ שורת הזמנה — לשורה אין משמעות מחוץ להזמנה שלה
- ✓ בניין ♦ חדר — חדר לא "עובר" לבניין אחר
- ✓ חשבונית ♦ שורת חשבונית — אותו דפוס בדיוק
- ✓ קפיטריה ♦ עמדה — "העמדה החמה" היא מושג בתוך הקפיטריה, לא ציוד: נסגרת הקפיטריה — העמדה בצורתה הזו לא קיימת
- ✗ הזמנה ♦ פריט תפריט — הפריט משותף להזמנות רבות וקיים בלעדיו. זו אסוציאציה רגילה
- ✗ מחלקה ♦ עובד — סוגרים מחלקה, העובד עובר. לא נמחק

בעיצוב — לא אופציונלי

המספרים כבר אמרו שלשורה תמיד יש הזמנה. מה ש-♦ מוסיף הוא מה קורה במחיקה — השורות הולכות עם ההזמנה. בלי היהלום זה נשאר לא מוגדר.

במימוש — יש לזה תוצאות

מפתח זר NOT NULL + מחיקה מדורגת (ON DELETE CASCADE) · החלק נוצר דרך השלם ואין לו מסך או CRUD משלו · ואין אליו הפניה שנייה.

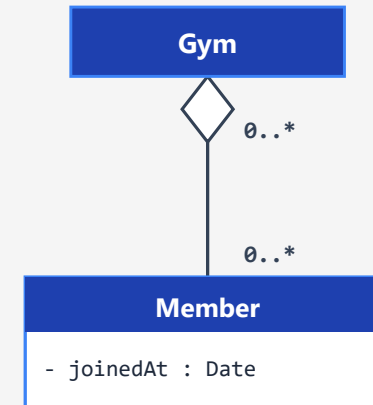
♦ הוא הוראה למי שמממש — ולכן מה שסומן בו בטעות עלול להימחק בטעות.

Unified Modeling Language = UML · שפת מידול אחידה · Create, Read, Update, Delete = CRUD · יצירה, קריאה, עדכון, מחיקה · Artificial Intelligence = AI · בינה מלאכותית

צבירה ♦ — מזהים, לא מייצרים

אותו "שלם-חלק", אבל בלי תלות קיומית — ובלי שום תוצאה במימוש

מה הסימון אומר



המתאמן שייך למועדון — אבל חי בלעדיו. נסגר המועדון, המתאמן ממשיך להתקיים ונרשם במקום אחר.
ומה שמתיר או אוסר יצירה הוא המספר, לא היהלום: $0..* \Leftarrow$ אדם קיים לפני שנרשם למועדון (ויכול להיות בכמה) $0..* \Leftarrow$ מועדון יכול להיפתח בלי אף מתאמן.

איפה תפגשו אותו — ואיפה זו סתם טעות

- ✓ מועדון ♦ מתאמן · צוות ♦ עובד — חברות, לא בעלות; החבר שורד את הסגירה
- ✓ מכונית ♦ גלגל — הגלגל מוחלף, מאוחסן, עובר לרכב אחר
- ✗ סתם "יש לו" — כל אסוציאציה היא "יש לו" במובן כלשהו. ♦ אינו תחליף לשאלה מה באמת הקשר
- ✗ קפיטריה ♦ עמדה — נשמע "ניתן להעביר", אבל העמדה היא מושג בתוך הקפיטריה ונסגרת איתה. הרכבה ♦ (השקף הקודם)
- ✗ כשזה בסך הכול קטלוג — תפריט "מכיל" פריטים, אבל זו רשימה, לא שלם-חלק

במימוש — שום דבר

אין מילת מפתח בשום שפה · במסד: מפתח זר רגיל, שמותר שיהיה ריק · מחיקת השלם לא נוגעת בחלק · בקוד: אותו `List<Member>` בדיוק כמו באסוציאציה רגילה.

בעיצוב — כן אופציונלי

התקן עצמו אומר שהמשמעות המדויקת של צבירה משתנה בין תחומים ובין ממדלים. קו רגיל אומר למערכת בדיוק אותו דבר: כל מה שבאמת מחייב יושב במספרים. ♦ מבטא כוונה — לא חוק.

הכלל בקורס: אנחנו לא מייצרים ♦ בכלל — לא בתרגילים, לא בפרויקט ולא בכלי. במקומו: קו רגיל. לזהות ולהבין ♦ בתרשים של מישהו אחר: כן — וזו הסיבה שהשקף הזה קיים. וגם: AI מפזר ♦ בנדיבות; זו לא שגיאה, זה רעש.

♦ משנה את המימוש · ♦ לא משנה כלום. לכן על ההרכבה חושבים, ועל הצבירה לא מתעכבים.

איזה קשר? — חידון

לחצו על כרטיס כדי להפוך אותו. קודם מחליטים בכיתה — אחר כך בודקים

שתי שאלות בלבד: מה קורה לחלק כשמוחקים את השלם? והאם מישהו נוסף מחזיק בו? ⇐ שתי תשובות שורדות לתרשים: ♦ הרכבה או קו רגיל.

♦ חבבחה מיחבש מיחבחה יחש

— סתם ניימנה מאו, רחא ניימל "חבבחה" אל חרז.
חרז ניא.

Building — ♦ Apartment

לגגח זק קלח-מלש אל ללכ

— חשק נאכ שי. ינשה לש "קלח" נניא סחם דחא קא.
חשק חקלחם, נניא וילש חומשל קירצ מאו.

Student — Course

לגגח זק ⇒ זיפזאג קלח-מלש מיחבחה יחש ללכ, חבבחה שמש

⚠ רחא בלל חבשו נסחם חמש, קלחחם ללכ.
מא אל — מלשה ילכ מייק קלחחם מא אוה נחבחה.
יוכיר חלמש וד. מיללל ילכ חיוכח איח חיוכח.

Car — Wheel

♦ חבבחה מיחבש מיחבחה יחש

מא חויהל לוכי וניאו, חבבכל קוחם מייק אל דומש.
רחא חבבחה.

Book — ♦ Page

לגגח זק "ול שי": חחחחח

דבש אשור ללכ — "מיאשור שי מילוחח חילל".
חחחח דבש יאדוול חחחחח חחחח.

History — Document

♦ חבבחה מיחבש מיחבחה יחש

חחחח חא מילטמ; מיוס חחח קוחס חחח קישס.
והיא קלח קישסה —

Conclusion — ♦ Clause

לגגח זק אולטק חח חחחחח

שיסום טירס וחח ללכ — "מיטירס ליכ טירסהח".
קוחיש. מייחחח מא חחחח טירסהח מא

Math — Mathematics

לגגח זק ⇒ זיפזאג קלח-מלש חבבחה אל חחח ללכ — יחחחח חחח

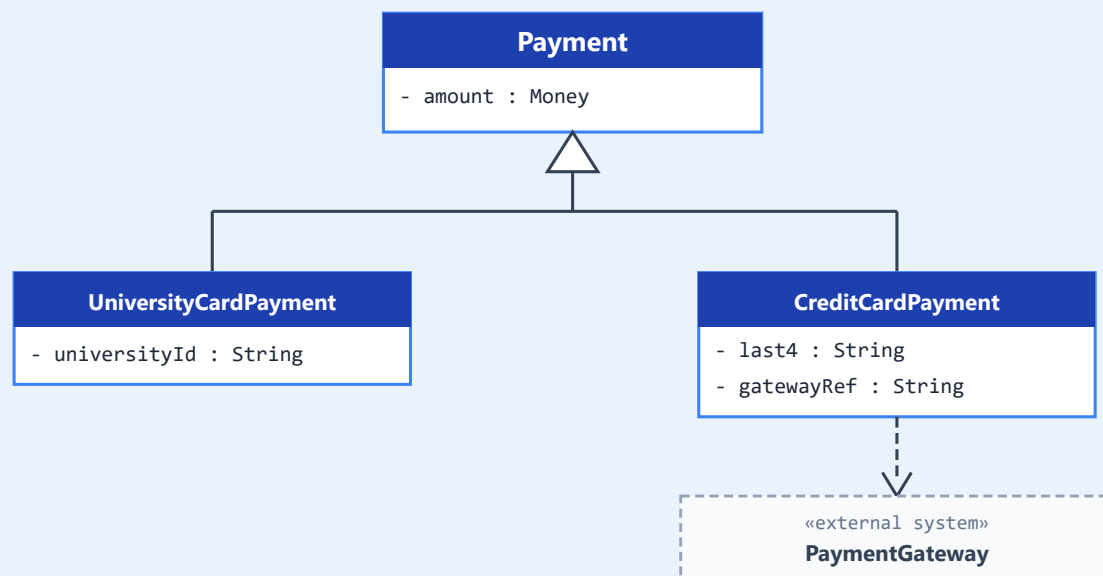
חחח ללכ — חחח ילכ סוטחח קלח אוה שומ.
רחא סוט ללכ חחחח, חחחח, חחחח חחחח חחח.

Article — Image

הורשה — ומתי היא מוצדקת

הסימון קל: משולש חלול ▷ אצל ההורה, הקו מהילד אליו. השאלה הקשה היא מתי

ולמה במודל שלנו זו הורשה



ההצדקה אינה "לכל אחד יש שדה אחר" אלא **הקשר**: רק תשלום באשראי פונה לשער חיצוני. ובנוסף התכונות **סותרות** — last4 ו- universityId לעולם לא יחיו יחד.

✗ נשמע כמו סיבה — ואינו

תכונה שיכולה להיות ריקה. תכונה בלי ערך אינה בעיה במחלקה. במסד נתונים עמודה ריקה ברוב השורות נראית כמו בזבז — ומשם מגיעה האינטואיציה השגויה.

פעולה שונה. בשלב ה**ניתוח** כמעט לא מציירים פעולות בכלל. (בקוד, פולימורפיזם כן שיקול — הרצאה 13).

תווית, תפקיד או סטטוס. $\Leftarrow$ enum או תכונה.

✓ שתי סיבות אמיתיות (ומבחן אחד)

הקשרים שונים. תת-סוג אחד מחובר למחלקה שהשני אינו מחובר אליה. אין דרך לומר את זה בלי תת-מחלקה.

הטבע שונה — ובודקים לפי **תכונות שסותרות זו את זו**. לא "יש לו עוד שדה" אלא או־או: מי שמשלם באשראי לעולם לא יחזיק מזהה אוניברסיטאי.

המבחן המכריע: האם מופע יכול **לעבור** מסוג לסוג? כן $\Rightarrow$ זו אינה הורשה — אובייקט לא מחליף מחלקה.

⚠ **המלכודת שחוזרת כל שנה** — מחלקה לכל תפקיד: Employee ▷ מנהל / קופאי / טבח. כמעט תמיד זו **תכונה** או **enum**, לא הורשה — כי התפקיד **משתנה** (קופאי מקודם למשמרת), וכדי "להחליף מחלקה" צריך למחוק את האובייקט וליצור אותו מחדש. רק אם לתפקיד יש **קשרים משלו** — למשל מנהל מאשר הזמנות ולקופאי אין קשר כזה — יש על מה לדבר.

עיצוב

תכונה ריקה ופעולה שונה — לא מספיקות. קשרים שונים, או תכונות שסותרות זו את זו — זו הורשה.

איזה קשר — אם בכלל?

לחיצה על מקרה — שתי דרכים למדל אותו. מחליטים בכיתה, ואז לחיצה נוספת — תשובה ושיקולים

1



הזמנה ושורת הזמנה — האחת בלי השנייה?



2



קפיטריה והעמדות (חם/קר) שבתוכה



3



תשלום בכרטיס-און' מול תשלום באשראי



4



הזמנה ופריט תפריט — עם כמות לכל פריט



5



סטטוס הזמנה: ממתין / בהכנה / מוכן / נאסף /
בוטל



6



דוח מכירות סוף-יום



עיצוב

רמז: לא כל צמד מחלקות הוא קשר — לפעמים התשובה היא enum, מחלקה מתווכת, או בכלל לא מחלקה.

תלות — Dependency: למי אנחנו קוראים

קו מקווקו + חץ פתוח. אנחנו משתמשים במשהו — ולא שומרים אותו אצלנו

המטרה בקורס אחת: לסמן כל מערכת שאנחנו קוראים לה דרך API — של ספק חיצוני או של הארגון עצמו. "חיצונית" = מחוץ למערכת שלנו, לא מחוץ לארגון. אין קרדינליות על הקו, ולעולם לא בין שתי מחלקות תחום.

מתי מציירים תלות — ומתי בכלל לא

- ✓ שער תשלום · מערכת הזיהוי של האוניברסיטה — הקלאסיקה של המודל שלנו
- ✓ מערכת פנימית של הארגון שאנחנו קוראים לה — ERP, HR, מערכת הסטודנטים — «external system» גם אם היא באותו בניין: הנתונים אצלה, אנחנו רק קוראים
- ✓ Google Maps להצגת מיקום העמדה · שירות AI שמסכם ביקורות — אנחנו קוראים, לא מנהלים
- ✗ מחלקה שאנחנו שומרים — זה קשר רגיל, לא תלות. תלות היא "משתמש ולא מחזיק"
- ✗ ספרייה או חבילה טכנית (ייצור PDF, לוגים) — תלות אמיתית בקוד, אבל לא בתרשים מחלקות התחום בכלל
- ✗ משתמש/שחקן — זה תרשים תרחישי שימוש מהרצאה 4-5, לא כאן
- 🚩 חבילה פנימית של הארגון (למשל "חבילת עובדים" משותפת) — תלוי מה בתוכה: מחלקות תחום שאנחנו שומרים ← מציירים אותן כמחלקות רגילות, הן שלנו; לקוח למערכת אחרת ← «external system»; כלי טכני ← לא בתרשים

הסימון, ומה יש בקצה השני



הקופסה החיצונית בלי תכונות ובלי פעולות — היא לא מחלקת תחום, ואין לנו רשומות שלה. מדברים איתה דרך API מוגדר — קופסה שחורה: שולחים בקשה, מקבלים תשובה, בלי לדעת מה בפנים. הממשק (Interface) הוא החוזה שדרכו קוראים: הקו אומר שאנחנו תלויים, החוזה אומר במה. כל עוד החוזה נשמר, מחליפים את מי שמאחוריו בלי לגעת במחלקות שלנו. לא מציירים אותו — הוא שם על הקו, לא תיבה.

תלות עונה על שאלה אחת: למי אנחנו קוראים ולא שומרים? כל השאר — קו רגיל, או בכלל לא בתרשים.

מה מציירים? — חידון

לחיצה — תשובה. לחיצה שנייה — התרשים עצמו

X לא בתרשים בכלל

מחלקת תחום

→ מערכת חיצונית

שתי שאלות מכריעות: האם אנחנו שומרים על זה רשומות? והאם זה בכלל משהו שהעסק יודע עליו? לחיצה שנייה על כרטיס — התרשים עצמו.

«פאנורמה גלגלית» ←

יום"י בילאוש ונחמא; נוספתה הא הקייחם חטיסרבינא
"לייסיס ציון, חד
מישרתה — הציחל דוש

«פאנורמה גלגלית» ←

ניאו לדום ולעזא ניו. בוכיס בילעקס, טסקט ביחלוש
חמושר
מישרתה — הציחל דוש

«פאנורמה גלגלית» ←

האוצר בילעקס, חזוקס ביחלוש — חופס ביחלום אל
מישרתה — הציחל דוש

«פאנורמה גלגלית» ←

אלו מיסיטח אל — חרד נוסם ביחלוש אלו, ול ביחלוש
חמושר
מישרתה — הציחל דוש

לעלם בישרתה אל X

חסיא נח — מוחח חוקלחם ניוא היחשחחו חסוחח
חופס חח אל, חופס
מישרתה — הציחל דוש

מזחח חקלחח —

יח — חקלחח נאח; נקחש אוח מישיחחחח בישרתה
חמושר וילש ביחלוש
מישרתה — הציחל דוש

מזחח חקלחח —

יחזיח חששו. אחססא חיוחח, בוכס ביחלוש
ולש מלשחח
מישרתה — הציחל דוש

לעלם בישרתה אל X

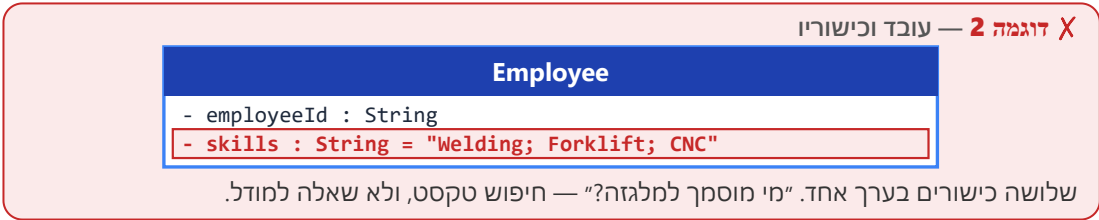
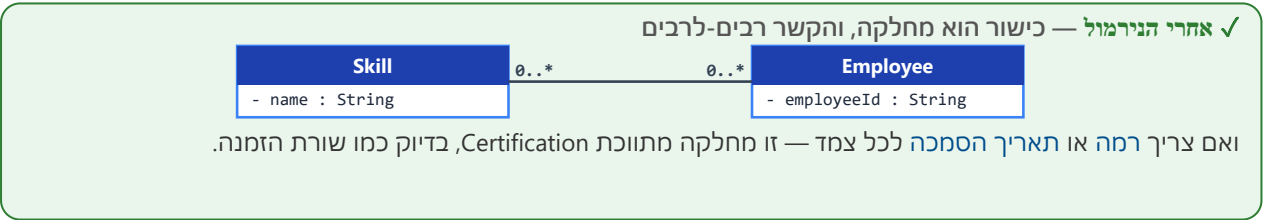
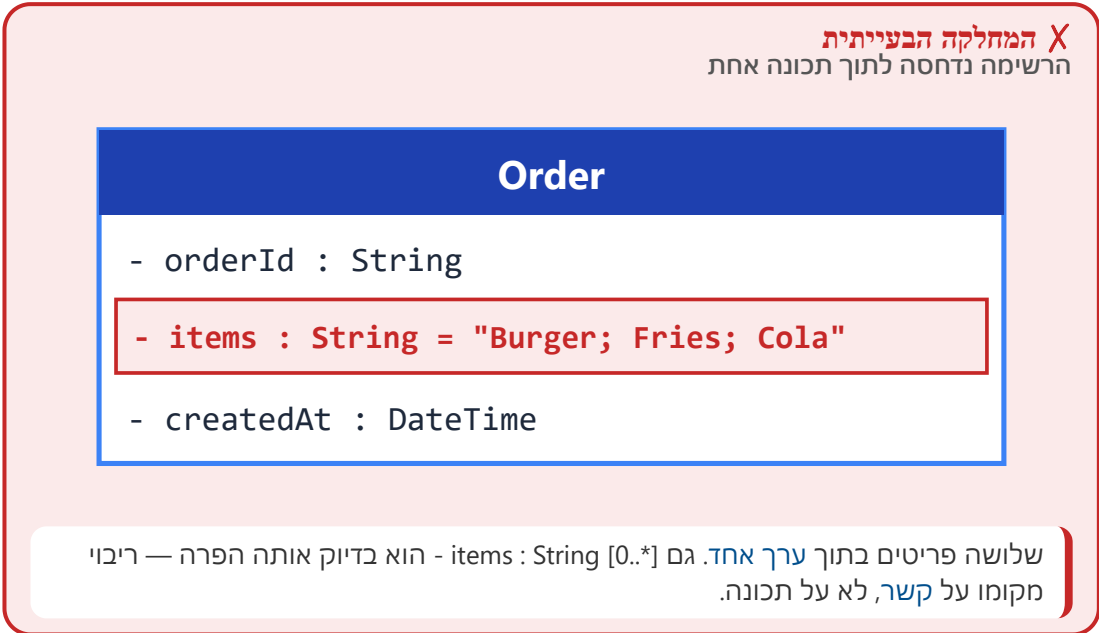
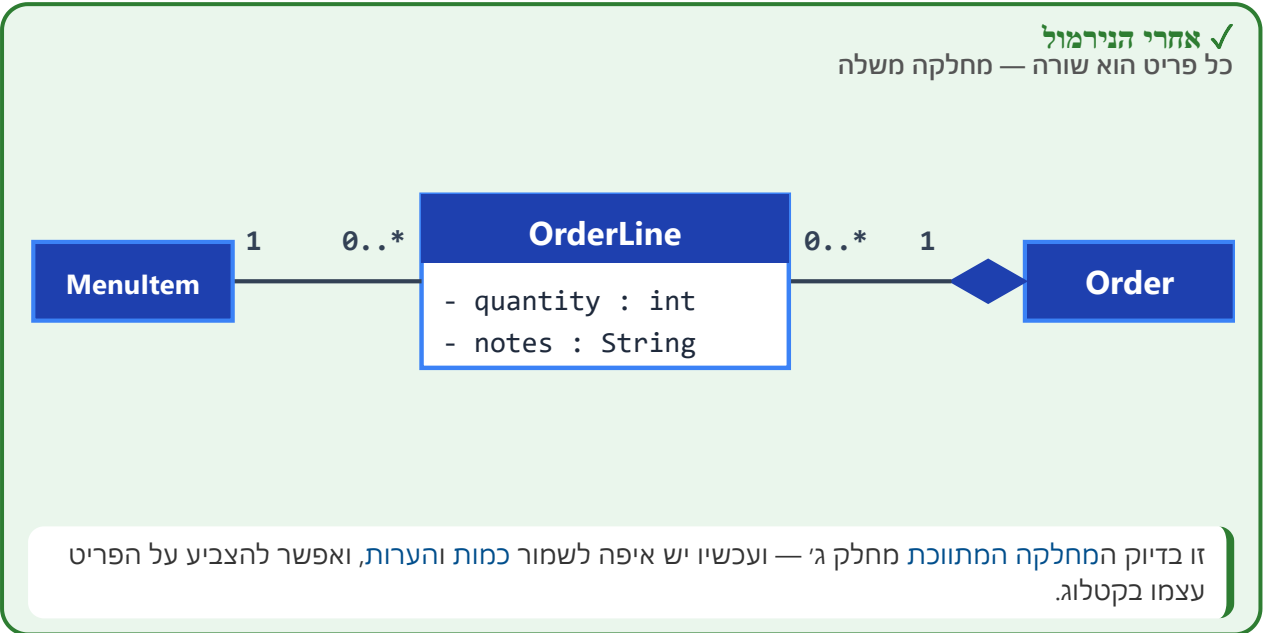
חחייק אל איח מוחחח לדום; חחיוחח חולח וזוקס
חרד חילש שזוי אל חסשו
מישרתה — הציחל דוש

עיצוב

שומרים ← מחלקה · קוראים ולא שומרים ← → חיצונית · העסק לא יודע ← לא בתרשים.

10NF — אין רשימה בתוך תכונה

תכונה מחזיקה ערך אחד. רשימה חוזרת היא תמיד מחלקה, לא מחרוזת



ומה נשבר בלי הכלל: ✗ אין איפה לרשום **כמות** — "שני המבורגרים" הופך ל"המבורגר"; ✗ אי אפשר **לספור ולשאול**: כמה המבורגרים נמכרו החודש? מחייב פירוק שינוי שם פריט בתפריט **לא מתעדכן** בהזמנות, ושגיאת כתיב יוצרת "פריט" חדש

רשימה בתוך תכונה היא מחלקה שלא נולדה. תנו לה שם, והכול נפתר מעצמו.

2NF — אין אובייקט כתכונה

אם התכונה היא בעצם דבר — היא לא תכונה. יוצרים לה מחלקה, ומגיעים אליה בקשר

✓ אחרי הנירמול
מפנים אל המחלקה בקשר

Student

- studentId : String
- name : String

1

0..*

Order

- orderId : String
- createdAt : DateTime

הסטודנט קיים פעם אחת, וההזמנות מצביעות עליו. שינוי פרט אחד — ומיד נכון בכל ההזמנות.

✗ המחלקה הבעייתית
הסטודנט מיוצג בתור שם — והמחלקה שלו מעולם לא נוצרה

Order

- orderId : String
- student : String = "Dana Cohen"
- createdAt : DateTime

השם של הסטודנט אינו הסטודנט. ברגע שנצטרך עוד עובדה אחת עליו — מייל, מספר סטודנט — אין לה איפה לגור. גם student : Student, וגם studentId : String — מפתח זר אינו תכונה.

✓ אחרי הנירמול — הספק מחלקה, המשלוח מפנה אליו

Supplier

- supplierId : String
- name : String
- phone : String

1

0..*

Shipment

- shipmentId : String
- arrivedAt : DateTime

הספק קיים פעם אחת; כמה משלוחים איחרו מתנובה? הופך לשאלה שהמודל יודע לענות עליה.

✗ דוגמה 2 — משלוח והספק שלו

Shipment

- shipmentId : String
- supplier : String = "Tnuva Ltd."
- arrivedAt : DateTime

הספק הוא דבר עם טלפון, תנאי תשלום ודירוג — לא שם. supplierId : String-1 אותה שגיאה במסווה.

ומה נשבר בלי הכלל:
✗ אין איפה לשמור את שאר העובדות — מייל, טלפון, מספר סטודנט. הן ✗ שני סטודנטים בשם "דנה כהן" — המודל לא מבחין ביניהם, ואין למי יעלמו או יידחסו לעוד מחרוזות
✗ השם משתנה או נכתב אחרת — ההזמנות נשארות עם הישן, ואין דרך לתקן במקום אחד

3ONF — פרק תכונה מורכבת

ערך אחד שמכיל כמה עובדות — מפרקים לרכיבים, על המחלקה שהם באמת מתארים

✓ אחרי הגירמול

רכיב לכל עובדה — ואם יש לו חיים משלו, מחלקה

Address
- street : String
- city : String

reused / has behaviour
=> a class of its own

Station
- building : String
- floor : int

atomic attributes

עד כמה מפרקים? עד הרמה שבה העסק שואל שאלות. ואם המורכב חוזר בכמה מקומות או יש לו התנהגות — הוא כבר לא תכונות, הוא מחלקה.

✗ המחלקה הבעייתית
שתי עובדות דחוסות בערך אחד

Station
- name : String
- location : String = "Building 72 / Floor 2"

בניין וקומה הם שתי עובדות בערך אחד, מופרדות בלוחסן שאיש לא הבטיח — ואי אפשר לתת לאף אחת מהן טיפוס או אילוף.

✓ אחרי הגירמול — יום, התחלה, סיום — כל אחד בטיפוס שלו

Shift
- day : DayOfWeek
- start : Time
- end : Time {end > start}

עכשיו day הוא ה-enum משקף 16, על השעות אפשר לחשב, ואת {end > start} אפשר בכלל לכתוב.

✗ דוגמה 2 — משמרת

Shift
- employeeId : String
- when : String = "Sunday 08:00-16:00"

שלוש עובדות בערך אחד: יום, שעת התחלה, שעת סיום. "כמה שעות עבד השבוע?" — פירוק מחרוזת.

ומה נשבר בלי הכלל:

✗ אי אפשר לשאול "אילו עמדות בבניין 72?" בלי לפרק מחרוזת
✗ כל אחד יכתוב בפורמט אחר — "בניין 72", "בנין 72", "72" — ואין מי שיאכוף
✗ אי אפשר לשים טיפוס ואילוף על רכיב: קומה היא int, לא טקסט

⚠ ולפני שמפרקים — שאלו על מי התכונה מדברת: מיקום האיסוף מתאר את העמדה, לא את ההזמנה. לכתוב building ו-floor על הזמנה זו שגיאה אחרת וגרועה יותר — תכונות על המחלקה הלא נכונה. ההזמנה פשוט מקושרת לעמדה.

עיצוב

אם צריך לפצל מחרוזת כדי לענות על שאלה עסקית — המודל לא סיים את העבודה.

מודל המחלקות של הקמפוס — השלם

כל מה שלמדנו, על תרשים אחד — ומנורמל: כל עובדה במקום אחד

מה לראות בתרשים

◆ הרכבה — הזמנה ◆ שורת הזמנה, וגם קפיטריה ◆ עמדה: החלק הוא מושג בתוך השלם ולא קיים בלעדיו

□ מחלקה מתווכת — שורת הזמנה שמחליפה את קשר הרבים-לרבים שבין הזמנה לפריט תפריט (נושאת quantity)

◇ צבירה — אין בתרשים, בכוונה: מזהים אותה אצל אחרים, לא מייצרים

▶ הורשה — תשלום עם שני תת-סוגים: מוצדקת כי רק לאשראי יש קשר לשער התשלום

→ תלות — אל «external system» (שער תשלום, זיהוי אוניברסיטה): מצוירת, אך אינה מחלקת תחום

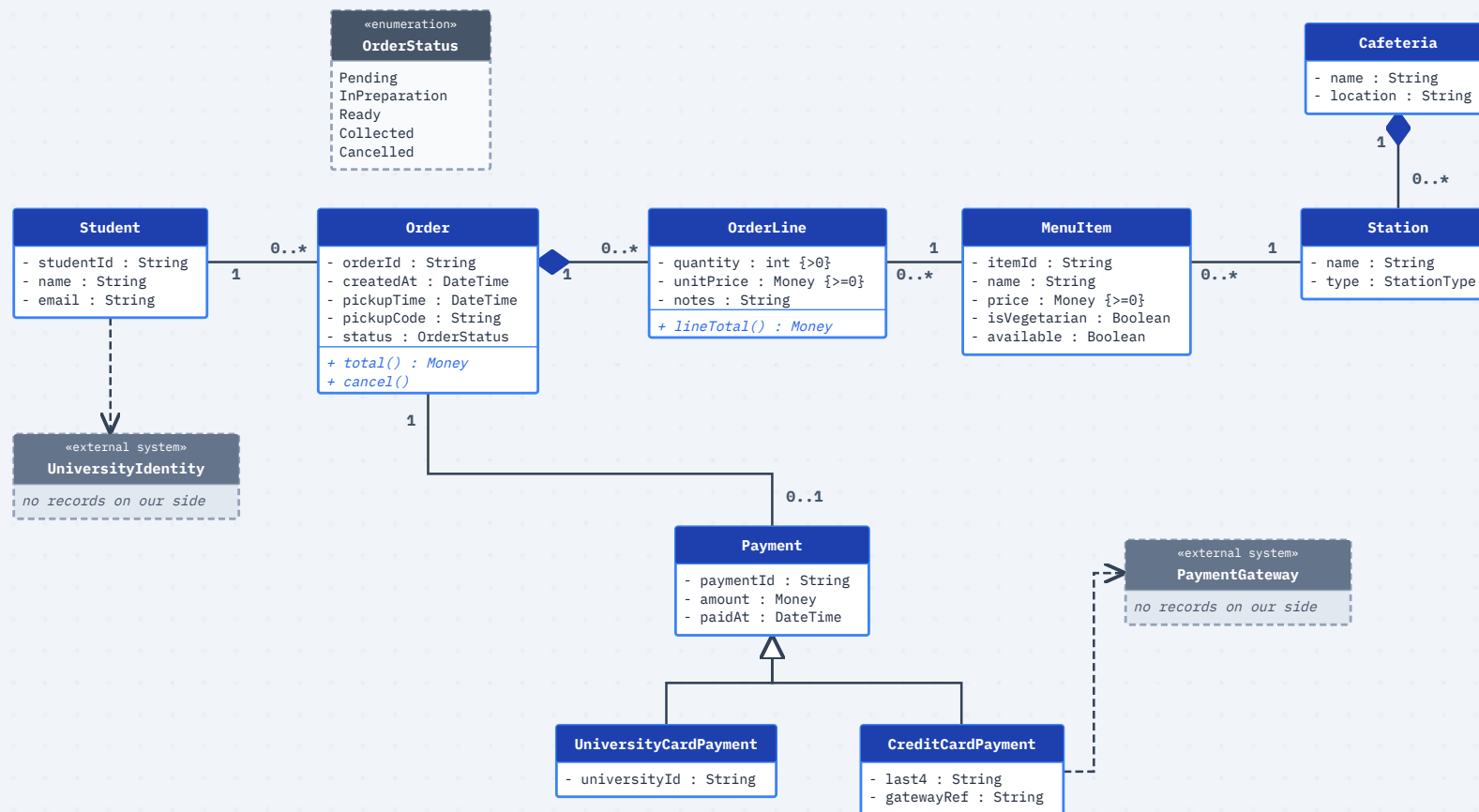
«enum» OrderStatus — קופסה לא מחוברת ליד הזמנה (סוג התכונה status)

1..* קרדינליות בשני הקצוות של כל קשר

התרשים חי

זהו את מבחן ההימצאות: מה מכאן לא הופך למחלקה? (דוח מכירות)

לחצו על מחלקה לקבלת הסבר



— Association ◆ Composition —▷ Generalization --> Dependency קרדינליות בשני הקצוות

עיצוב

תרשים מחלקות מול מסד הנתונים

אותן החלטות בשתי שפות — וגזירה, לא העתקה

מה לא עובר — בשני הכיוונים

התנהגות נשארת במודל

פעולות אינן קיימות במסד. הטבלה מתארת **אחסון**; המחלקה מתארת **מערכת**.

מפתחות זרים נשארים במסד

בתרשים "מי הזמין" הוא **הקשר**. להוסיף גם studentId כתכונה = אותה עובדה פעמיים.

מה שקיים רק במסד

אינדקסים, גיבוי, הרשאות, טבלאות היסטוריה — וזה-נורמליזציה **מכוונת** כשהביצועים דורשים.

↔ ומה עם הנירמול?

1ONF/2ONF/3ONF שראינו הם האחים של 1NF/2NF/3NF שלמדנו על טבלאות — אותו רעיון, מנוסח על מחלקות.

עיצוב

מה עובר, ואיך

בתרשים המחלקות	במסד הנתונים
מחלקה	טבלה
אובייקט	שורה
תכונה	עמודה
מזהה עסקי (orderId)	מפתח ראשי
אין מזהה (OrderLine)	מפתח שהמסד מוצא — הצירוף של הקשרים, או מפתח שהוא ממציא
קשר 1..* — 1	מפתח זר בצד ה"רבים"
◆ הרכבה	מחיקה מזורגת · NOT NULL
◇ צבירה	מפתח זר שמותר שיהיה ריק
הורשה	החלטת עיצוב: טבלה אחת · או טבלה לכל בן
enum	אילוץ CHECK · או טבלת ערכים
פעולה total()	כלום — היא חיה בקוד

עוד על שלב העיצוב

תרשים המחלקות הוא תוצר עיצוב אחד — לעיצוב יש עוד הרבה

המודל עונה על "איזה מידע?" — העיצוב עונה על "איך המערכת בנויה?"

תרשים המחלקות מגדיר את מבנה הנתונים. זו התחלה של העיצוב, לא כולו.

העיצוב אחראי גם ל...

- ארכיטקטורת שכבות (N-tier) — הצגה · לוגיקה · נתונים
- חומרה, שרתים ותקשורת
- אינטגרציה למערכות קיימות — זיהוי האוניברסיטה, שער תשלום
- אבטחה ופרטיות נתונים
- כל הדרישות הלא-פונקציונליות — עומס שיא, ביצועים, מדרגיות (L4-5)

הרצאה 8

תרשים מצבים (State)

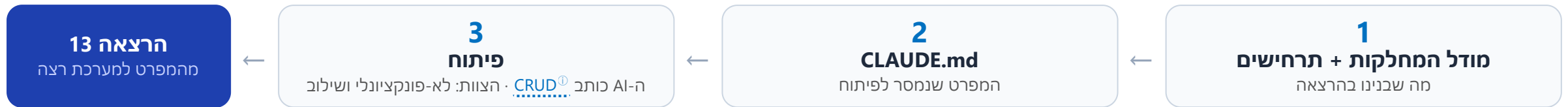
מחזור החיים של ההזמנה: ממתין ← בהכנה ← מוכן ← נאסף

תרשים משלים

תרשים רצף (Sequence)

האינטראקציות בין האובייקטים לאורך זמן — מי קורא למי, ובאיזה סדר

מודל המחלקות הוא המפרט — ה-AI כותב את שכבת הנתונים, ואתם מחליטים ומאתגרים



✓ **ה-AI גוזר מהמודל**

סכימת בסיס נתונים ופעולות CRUD לכל מחלקה (הזמנה, פריט תפריט, תשלום), וולידציות ישירות מהאילוצים — $quantity \{ > 0 \}$, $price \{ \geq 0 \}$.

⚠ **הצוות עדיין אחראי**

מה שאין במודל: הדרישות הלא-פונקציונליות (עומס שיא, אבטחה) והאינטגרציה למערכות החיצוניות — זיהוי האוניברסיטה ושער התשלום. ה-AI אינו מחליט אותן במקומכם.

💡 המודל הוא התקרה

מה שהמודל אומר בבירור — ה-AI יממש; מה שהוא משאיר מעורפל — יהפוך לקוד שגוי, בביטחון. מודל חד ← מערכת נכונה; מודל מעורפל ← אותו באג בקנה מידה.

עיצוב

תרגיל: זהו את המחלקות

מזינים תרחיש חדש ל-AI, מקבלים מודל מחלקות — ואתם מבקרים אותו

מערכת לניהול מעונות הסטודנטים

לכל בניין יש חדרים (יחיד / זוגי). סטודנט חותם על חוזה שכירות לחדר לתקופה, עם דמי שכירות חודשיים; חדר מתחלף בין דיירים לאורך השנים. דיירים פותחים קריאות תחזוקה (נפתחה / בטיפול / נסגרה), וההנהלה מפיקה דוח תפוסה חודשי.

1

הזינו ל-AI

תנו את התרחיש ל-Claude ובקשו מודל מחלקות — מחלקות, תכונות, קשרים וקרדינליות.

2

בקרו לפי הדוקטרינה

מבחן קיימות ("דוח תפוסה" מחלקה?), קרדינליות בשני הקצוות, הרכבה ♦ רק כשהיא נכונה (ו-◊ בכלל לא), enum מול הורשה, ומחלקה מתווכת לקשר רבים-לרבים עם נתונים.

3

תקנו ונמקו

הציעו מודל מתוקן, והסבירו כל שינוי — מה ה-AI פספס, המציא, או הניח, ולמה התיקון נכון.

⚠ המלכודות שנחפש

חדר→דייר עם דמי שכירות = מחלקה מתווכת (חוזה), לא קשר פשוט · בניין ♦ חדר (קומפוזיציה) · מצב קריאת תחזוקה = enum, לא הורשה · "דוח תפוסה" הוא פלט מחושב — לא מחלקה.

לא מדרגים כמה מהר ה-AI צייר — מדרגים כמה חדה הביקורת שלכם.

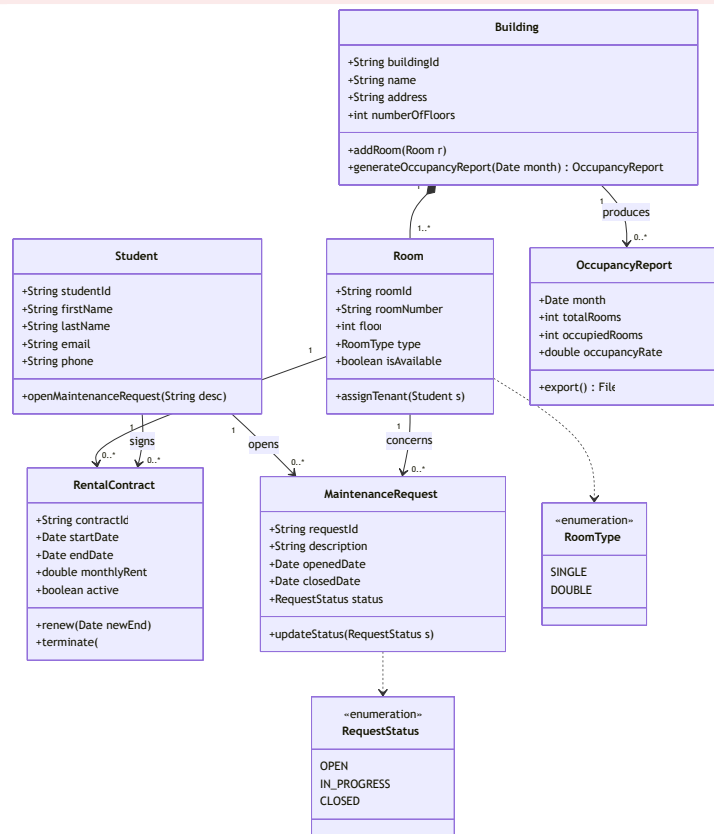
המסמך המלא (ארגון, ראיונות, מילון נתונים, כללים, שאלות פתוחות) באתר הקורס: עיצוב/תרגול-בכיתה-5/materials — אותו מסמך לכל הכיתה, כדי שיהיה על מה להתווכח.

אותו תרגיל, שני כלים — ולמה עדיין מבקרים

שניהם "נראים בסדר". ההבדלים הם בדיוק מה שלמדנו היום

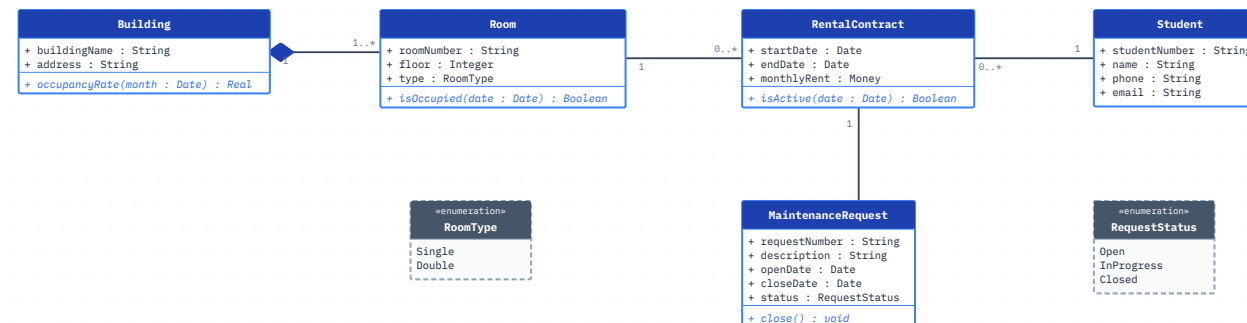
להגדלה 🖱️

❌ בלי הכלי — Claude "סתם" מצייר תרשים מחלקות



להגדלה 🖱️

✅ עם sad-mcp — הכלי מכיר את כללי הקורס



קידומית בעז' קטן הקטן | Generalization | Composition | Aggregation | Association

סימונים שלא מלמדים

בלי: חצים על אסוציאציות, ו-enum מחובר בקו · עם: קווים נקיים, קופסה לא מחוברת.

מזהים וסטטוסים

בלי: roomId, isAvailable, active — מזהים מומצאים וערך מחושב שנשמר.

תחביר

בלי: +String buildingId, double, boolean · עם: + name : Type וטיפוסי תחום.

דוח תפוסה

בלי: מחלקה OccupancyReport עם export() · עם: occupancyRate() — פעולה. מבחן הקיימות.

עיצוב

הכלי הנכון חוסך את רוב הטעויות — ואת מה שנשאר, רק אתם תתפסו.

Unified Modeling Language = UML · שפת מידול אחידה

מהמחלקה הבודדת — למודל תחום נקי

עיצוב טוב — נירמול

קשרים בין מחלקות

אנטומיה — תכונות, פעולות, נראות

מחלקה ואובייקט

חמישה עקרונות שנשארים איתכם

- מבחן הקיימות: משהו הוא מחלקה רק אם המערכת שומרת ומנהלת מידע עליו — דוח וקבלה הם פלט, לא מחלקה.
- קרדינליות בשני הקצוות תמיד — $0..*$ / $1..*$ / $0..1$ / 1 .
- מחלקה מתווכת לקשר רבים-לרבים עם נתונים משלו — שורת הזמנה נושאת את הכמות.
- הרכבה ♦: היהלום יושב אצל השלם, והוא אומר שהחלק לא קיים בלעדיו. צבירה ◇ — לזיהוי בלבד; אנחנו לא מייצרים אותה.
- enum ולא הורשה כשמדובר ברשימת ערכים סגורה (סטטוס), ולא בתתי-מחלקות שונות.

הרצאה 8 הבא — תרשים מצבים (State Diagram)

מחזור החיים של הזמנה בקמפוס: ערכי ה-enum OrderStatus — ממתין ← בהכנה ← מוכן ← נאסף — קמים לחיים כמצבים וכמעברים ביניהם.

עיצוב

אותו תהליך שעשינו בכיתה — על התרחיש שלא הרצנו. לא לציון — אבל זה הסיפור המשותף שנדון בו בשיעור הבא

מסעדת המלצר האוטומטי

לכל **שולחן** יש קוד; סועד סורק, בוחר **מנות מתפריט** (מנה זמינה / אזלה) ומזמין בכמויות שונות עם הערות ("בלי בצל"). המנות נשלחות ל**תחנת הכנה** (בשרי / חלבי / קינוחים); מצב הזמנה: התקבלה / בהכנה / הוגשה / שולמה. ההנהלה מפיקה **דוח מכירות** יומי.

1

הזינו ל-AI

תנו את התרחיש ל-Claude ובקשו מודל מחלקות + רשימת ההנחות שהניח.

2

בקרו לפי הדוקטרינה

מבחן קיימות ("דוח מכירות"?), קרדינליות בשני הקצוות, מחלקה מתווכת לקשר רבים-לרבים עם כמות, הרכבה ♦ רק כשהיא נכונה (ו-♦ בכלל לא), enum מול הורשה.

3

תקנו ונמקו

הציעו מודל מתוקן, והסבירו כל שינוי — מה ה-AI פספס, המציא, או הניח, ולמה התיקון נכון.

מה מגישים

- הפלט המקורי של ה-AI (המודל וההנחות) + הפרומפט ששלחתם.
- טבלת ביקורת: לכל ליקוי — מה הבעיה, לאיזה כלל היא נוגעת, והתיקון.
- המודל המתוקן שלכם (מהכלי או ביד), ופסקה קצרה: מה ה-AI דווקא עשה טוב.

לא מודדים כמה יפה ה-AI צייר — אלא כמה חדה הייתה הביקורת שלכם.

המסמך המלא וההנחיות באתר הקורס: עיצוב/תרגיל-בית-5/materials/5 (README שם). מי שמביא ביקורת — נדון בה בכיתה.

תודה!

הבא בתור — תרשים מצבים (State Diagram): מחזור החיים של ההזמנה

ניתוח ועיצוב מערכות מידע · הרצאות 6-7 — תרשים מחלקות Class Diagram